UNIVERSIDADE SÃO FRANCISCO

GUILHERME HENRIQUE DE SOUZA RAMOS – RA: 202319276
JÚLIO CESAR GRILO DOS SANTOS – RA: 202641939
PABLO PERICLES DE ASSIS SOUZA – RA: 202239733
THIAGO ALDEIA DUARTE – RA: 202615867

PÁGINAS EM ORDEM

SISTEMA DE GERENCIAMENTO DE BIBLIOTECA EM LINGUAGEM C

BRAGANÇA PAULISTA 2026

UNIVERSIDADE SÃO FRANCISCO

GUILHERME HENRIQUE DE SOUZA RAMOS
JÚLIO CESAR GRILO DOS SANTOS
PABLO PERICLES DE ASSIS SOUZA
THIAGO ALDEIA DUARTE

PÁGINAS EM ORDEM

SISTEMA DE GERENCIAMENTO DE BIBLIOTECA EM LINGUAGEM C

Trabalho avaliativo apresentado à Universidade São Francisco como requisito para aprovação na disciplina EP50283 – Prática e Tecnologia: Ferramentas e Técnicas de Programação.

Docente: Pedro Fascina Casarina

BRAGANÇA PAULISTA 2026

RESUMO

Bibliotecas escolares, comunitárias e de pequeno porte ainda controlam, em muitos casos, o acervo e os empréstimos em cadernos ou planilhas, o que dificulta saber quais livros estão disponíveis, quem está com cada exemplar e desde quando. Diante desse problema, este projeto teve como objetivo desenvolver, em linguagem C, um sistema de gerenciamento de biblioteca executado no console, capaz de cadastrar, consultar, alterar e excluir livros, autores, editoras e usuários, além de registrar empréstimos e devoluções. O método adotado foi o desenvolvimento incremental em dez etapas: definição dos requisitos, modelagem de cinco estruturas (struct), construção dos menus, implementação das operações CRUD, módulo de empréstimos, persistência em arquivos binários, integração, testes e documentação. Como funcionalidade adicional, o sistema verifica a disponibilidade de exemplares antes de cada empréstimo, atualiza automaticamente a quantidade disponível na retirada e na devolução, registra as datas pelo relógio do computador e impede exclusões que deixariam dados inconsistentes. O resultado é uma aplicação funcional, modularizada em funções e com validação das entradas numéricas, que mantém os dados salvos entre execuções e demonstra na prática o uso de structs, vetores, funções, estruturas de decisão e repetição e manipulação de arquivos.

Palavras-chave: linguagem C; CRUD; biblioteca; empréstimos; structs.

SUMÁRIO

1 INTRODUÇÃO 5
2 OBJETIVOS 5
3 RECURSOS E MÉTODOS 6
4 RESULTADOS ESPERADOS 7
5 PLANEJAMENTO DO SOFTWARE 7
6 ESTRUTURAS DE DADOS 10
7 DEFINIÇÃO DE CADA FUNÇÃO 13
8 FUNCIONALIDADE EXTRA: EMPRÉSTIMOS E DEVOLUÇÕES 16
9 VALIDAÇÃO DE DADOS E TRATAMENTO DE ERROS 17
10 DESCRIÇÃO DO PROJETO E INTERFACE DO USUÁRIO 18
11 CONSIDERAÇÕES FINAIS 30
REFERÊNCIAS 31
APÊNDICE A – CÓDIGO DO PROJETO 32

1 INTRODUÇÃO

A informatização do controle de informações tornou-se indispensável em praticamente todas as organizações, e as bibliotecas não são exceção. Uma biblioteca precisa saber, a qualquer momento, quais títulos possui, quantos exemplares de cada um estão na estante, quais foram emprestados, para quem e em que data. Quando esse controle é feito em fichas de papel, cadernos ou planilhas soltas, surgem problemas conhecidos: exemplares que “somem”, empréstimos que ninguém lembra de cobrar, cadastros duplicados e dificuldade para responder a uma pergunta simples como “esse livro está disponível?”.

Esse cenário é comum em bibliotecas escolares, comunitárias e de pequenas instituições, que em geral não dispõem de recursos para adquirir sistemas comerciais de gestão de acervo. No Brasil, a Lei nº 12.244, de 2010, determinou a universalização das bibliotecas nas instituições de ensino, o que reforça a necessidade de ferramentas simples e acessíveis para organizar esses espaços (BRASIL, 2010). Em cidades do porte de Bragança Paulista, onde está a Universidade São Francisco, escolas e projetos comunitários são exemplos de locais que poderiam se beneficiar de uma solução leve, que funcione em qualquer computador e não dependa de internet.

Este projeto propõe o desenvolvimento de um Sistema de Gerenciamento de Biblioteca em linguagem C, executado no console, com operações de cadastro, consulta, alteração e exclusão (CRUD) de livros, autores, editoras e usuários, além do controle de empréstimos e devoluções. O sistema relaciona as informações entre si — cada livro aponta para seu autor e sua editora, e cada empréstimo aponta para um usuário e um livro — e grava todos os dados em arquivos, para que nada se perca quando o programa é fechado.

Do ponto de vista acadêmico, a proposta é relevante porque obriga o grupo a aplicar, em um problema real e de ponta a ponta, os conteúdos centrais da disciplina: estruturas (`struct`), vetores, funções, estruturas de decisão e repetição, validação de entradas e manipulação de arquivos. Para o curso e para a universidade, o trabalho dialoga com a proposta extensionista da disciplina, ao aproximar o conhecimento técnico de uma necessidade concreta da comunidade: organizar e democratizar o acesso aos livros.

2 OBJETIVOS

2.1 Objetivo geral

Desenvolver, em linguagem C, um sistema de gerenciamento de biblioteca que permita cadastrar, consultar, alterar e excluir livros, autores, editoras e usuários e controlar empréstimos e devoluções, para organizar o acervo e o atendimento de bibliotecas escolares, comunitárias e de pequeno porte que ainda fazem esse controle de forma manual.

2.2 Objetivos específicos

- Modelar as entidades do sistema (Livro, Autor, Editora, Usuário e Empréstimo) por meio de cinco estruturas (`struct`) relacionadas entre si por códigos.
- Implementar as operações CRUD completas para livros, autores, editoras e usuários, com menus organizados por entidade.
- Implementar o registro de empréstimos e devoluções, verificando a disponibilidade dos exemplares e atualizando automaticamente a quantidade disponível.
- Garantir a integridade dos dados, impedindo exclusões que deixariam registros "órfãos".
- Validar as entradas do usuário e tratar erros de forma simples, sem que o programa trave.
- Persistir os dados em arquivos binários, mantendo as informações entre uma execução e outra.

3 RECURSOS E MÉTODOS

3.1 Recursos utilizados

O sistema foi desenvolvido em linguagem C padrão, usando apenas as bibliotecas da linguagem (`stdio.h`, `stdlib.h`, `string.h` e `time.h`), sem dependências externas. As ferramentas empregadas estão listadas na Tabela 1.

Tabela 1 – Ferramentas utilizadas no desenvolvimento

Ferramenta	Uso no projeto
Visual Studio Code	Editor de código, com extensão para compilação e depuração de programas em C.
GCC (MinGW no Windows)	Compilador C, usado com as opções de aviso `-Wall` e `-Wextra` para detectar problemas no código.
Git e GitHub	Controle de versão e compartilhamento do código entre os integrantes do grupo.
Microsoft Word	Elaboração da documentação do projeto e exportação para PDF.

Fonte: elaborado pelos autores (2026).

3.2 Método de desenvolvimento

O trabalho seguiu um desenvolvimento incremental: cada parte do sistema foi construída, compilada e testada antes da próxima ser iniciada. Primeiro foram definidas as estruturas de dados, apresentadas na entrega parcial. Em seguida foram criados os menus e as operações CRUD de cada entidade, começando pelas mais simples (Autor e Editora) e seguindo para as que dependem delas (Livro e Usuário). Depois foi implementado o módulo

de empréstimos e devoluções, que liga usuários e livros, e por fim a gravação e leitura dos arquivos.

O código foi organizado em funções pequenas, cada uma com uma responsabilidade clara (ler dados, buscar um registro, exibir uma tabela, cadastrar, alterar, excluir, salvar, carregar), e documentado com comentários que explicam o papel de cada bloco. O versionamento foi feito no GitHub, no repositório `github.com/JulioCesarGrilo/Gerenciador-de-biblioteca`, o que permitiu registrar as alterações e as correções de erros ao longo do projeto.

Os testes foram feitos executando o programa com roteiros de uso: cadastro de autores, editoras, livros e usuários; consultas; alterações; exclusões permitidas e bloqueadas; empréstimos até esgotar os exemplares; devoluções; e digitação de valores inválidos nos menus. As telas da seção 10 foram obtidas a partir desses roteiros.

4 RESULTADOS ESPERADOS

Espera-se, ao final do projeto, um Sistema de Gerenciamento de Biblioteca funcional e organizado, capaz de facilitar o controle das principais informações de uma biblioteca. Na prática, isso significa que o responsável pela biblioteca poderá:

- saber, a qualquer momento, quantos exemplares de cada livro existem e quantos estão disponíveis;
- registrar um empréstimo em poucos passos, sem risco de emprestar um livro que não está na estante;
- consultar o histórico de empréstimos, com as datas de retirada e devolução e a situação de cada um;
- manter os cadastros atualizados sem perder informações ao fechar o programa;
- confiar na consistência dos dados, pois o sistema impede a exclusão de registros ainda em uso.

Do ponto de vista acadêmico, espera-se que o projeto demonstre a capacidade do grupo de aplicar de forma integrada os conceitos da linguagem C — `struct`, funções, vetores, estruturas de decisão e repetição e manipulação de arquivos — na construção de uma solução completa para um problema real.

5 PLANEJAMENTO DO SOFTWARE

O desenvolvimento foi dividido nas dez etapas definidas na entrega parcial. A Tabela 2 mostra cada etapa e a situação no momento desta entrega final.

Tabela 2 – Etapas do desenvolvimento

Etapa	Descrição	Situação
1	Definição dos requisitos do sistema	Concluída
2	Criação das structs	Concluída
3	Desenvolvimento dos menus	Concluída
4	Implementação dos CRUDs	Concluída
5	Implementação do sistema de empréstimos e devoluções	Concluída
6	Implementação da gravação e leitura de arquivos	Concluída
7	Integração dos módulos	Concluída
8	Testes e correção de erros	Concluída
9	Organização e documentação do código	Concluída
10	Apresentação do projeto final	Esta entrega

Fonte: elaborado pelos autores (2026).

5.1 Organização em módulos

O código-fonte foi organizado em módulos lógicos dentro do arquivo `projetocrud.c`, cada um formado por um grupo de funções:

- **Utilitários:** leitura segura de texto e de números, limpeza de tela, pausa e obtenção da data atual.
- **Busca:** localizam um registro pelo código e devolvem sua posição no vetor.
- **Exibição:** imprimem as tabelas de cada entidade, reaproveitadas pelas telas de consulta, alteração e exclusão.
- **CRUD:** cadastrar, consultar, alterar e excluir para Autor, Editora, Livro e Usuário.
- **Empréstimos e devoluções:** funcionalidade extra, que relaciona usuários e livros.
- **Persistência:** gravação e leitura dos arquivos binários.
- **Menus:** menu principal e um submenu para cada entidade.

Os registros ficam em vetores globais de tamanho fixo, acompanhados de um contador que indica quantas posições estão ocupadas. Os limites e os nomes dos arquivos são definidos por constantes (`#define`), listadas na Tabela 3.

Tabela 3 – Constantes do sistema

Constante	Valor	Significado
`MAX_LIVROS`	200	Número máximo de livros
`MAX_AUTORES`	100	Número máximo de autores

MAX_EDITORAS	100	Número máximo de editoras
MAX_USUARIOS	200	Número máximo de usuários
MAX_EMPRESTIMOS	500	Número máximo de empréstimos
ARQ_LIVROS ... ARQ_EMPRESTIMOS	"livros.dat" ... "emprestimos.dat"	Arquivos binários de cada entidade
STATUS_EMPRESTADO	0	Empréstimo em aberto
STATUS_DEVOLVIDO	1	Empréstimo encerrado

Fonte: elaborado pelos autores (2026).

6 ESTRUTURAS DE DADOS

O modelo de dados do sistema é formado por cinco estruturas (`struct`), mantidas exatamente como foram apresentadas na entrega parcial. Cada estrutura agrupa as informações de uma entidade da biblioteca. As estruturas não guardam cópias umas das outras: elas se relacionam apenas pelo código, como em um banco de dados relacional.

6.1 Struct Livro

Representa um título do acervo. É a estrutura com mais campos (sete) e a mais importante do sistema, pois se liga ao autor, à editora e aos empréstimos.

```
struct Livro {
    int codigo;
char titulo[100];
int codigoAutor;
int codigoEditora;
    int ano;     int
quantidade;     int
quantidadeDisponivel;
};
```

Tabela 4 – Campos da struct Livro

Campo	Tipo	Descrição
`codigo`	int	Identificador único do livro, gerado automaticamente.
`titulo`	char[100]	Título do livro.
`codigoAutor`	int	Código do autor do livro (ligação com a struct Autor).
`codigoEditora`	int	Código da editora do livro (ligação com a struct Editora).
`ano`	int	Ano de publicação.
`quantidade`	int	Total de exemplares que a biblioteca possui.
`quantidadeDisponivel`	int	Exemplares que não estão emprestados; atualizado automaticamente a cada empréstimo e devolução.

Fonte: elaborado pelos autores (2026).

6.2 Struct Autor

```
struct Autor {
    int codigo;     char
nome[100];     char
nacionalidade[50];
};
```

Tabela 5 – Campos da struct Autor

Campo	Tipo	Descrição
codigo	int	Identificador único do autor, gerado automaticamente.
nome	char[100]	Nome do autor.
nacionalidade	char[50]	Nacionalidade do autor.

Fonte: elaborado pelos autores (2026).

6.3 Struct Editora

```
struct Editora {
    int codigo;
char nome[100];
char cidade[50];
};
```

Tabela 6 – Campos da struct Editora

Campo	Tipo	Descrição
codigo	int	Identificador único da editora, gerado automaticamente.
nome	char[100]	Nome da editora.
cidade	char[50]	Cidade da editora.

Fonte: elaborado pelos autores (2026).

6.4 Struct Usuario

```
struct Usuario {
    int codigo;
char nome[100];
char cpf[15];
char telefone[20];
};
```

Tabela 7 – Campos da struct Usuario

Campo	Tipo	Descrição
codigo	int	Identificador único do usuário (leitor), gerado automaticamente.
nome	char[100]	Nome do usuário.
cpf	char[15]	CPF, guardado como texto para aceitar a pontuação (000.000.000-00 tem 14 caracteres + terminador).
telefone	char[20]	Telefone de contato, guardado como texto para aceitar parênteses, espaço e hífen.

Fonte: elaborado pelos autores (2026).

6.5 Struct Emprestimo

```
struct Emprestimo {
```

```
    int codigo;     int
codigoUsuario;     int
codigoLivro;     char
dataEmprestimo[11];
char dataDevolucao[11];
    int status;
};
```

Tabela 8 – Campos da struct Emprestimo

Campo	Tipo	Descrição
`codigo`	int	Número do empréstimo, gerado automaticamente.
`codigoUsuario`	int	Código do usuário que retirou o livro (ligação com a struct Usuario).
`codigoLivro`	int	Código do livro emprestado (ligação com a struct Livro).
`dataEmprestimo`	char[11]	Data da retirada no formato DD/MM/AAAA (10 caracteres + terminador), preenchida automaticamente.
`dataDevolucao`	char[11]	Data da devolução; fica vazia até o livro ser devolvido.
`status`	int	`STATUS_EMPRESTADO` (0) ou `STATUS_DEVOLVIDO` (1).

Fonte: elaborado pelos autores (2026).

6.6 Relacionamentos entre as estruturas

A Tabela 9 resume como as estruturas se ligam. Quando o sistema precisa mostrar o nome do autor de um livro, por exemplo, ele pega o `codigoAutor` do livro e procura esse código no vetor de autores.

Tabela 9 – Relacionamentos entre as structs

Campo	Referência	Uso no sistema
`Livro.codigoAutor`	`Autor.codigo`	Exibir o nome do autor nos detalhes do livro; impedir a exclusão de autor com livros.
`Livro.codigoEditora`	`Editora.codigo`	Exibir o nome da editora nos detalhes do livro; impedir a exclusão de editora com livros.
`Emprestimo.codigoUsuario`	`Usuario.codigo`	Exibir o nome do usuário no histórico; impedir a exclusão de usuário com empréstimo em aberto.
`Emprestimo.codigoLivro`	`Livro.codigo`	Exibir o título no histórico; atualizar a quantidade disponível; impedir a exclusão de livro emprestado.

Fonte: elaborado pelos autores (2026).

7 DEFINIÇÃO DE CADA FUNÇÃO

Esta seção descreve o papel de cada função do código-fonte, agrupadas pelos módulos apresentados na seção 5.1. Todas as funções têm protótipo declarado no início do arquivo, o que permite que uma função chame outra escrita mais abaixo.

7.1 Funções utilitárias

- **lerLinha:** lê uma linha digitada pelo usuário com `fgets`, que aceita espaços e respeita o tamanho do vetor de destino. Remove o `\n` do final e, se o texto digitado for maior que o espaço disponível, descarta o restante da linha para que ele não seja lido por engano na pergunta seguinte. Se a entrada terminar de forma inesperada, devolve um texto vazio.
- **lerInteiro:** mostra uma mensagem, lê a resposta como texto e converte para número com `strtol`. Se nada for convertido ou se sobrar algum caractere depois do número (por exemplo, “12a”), exibe “Valor invalido. Digite somente numeros.” e pergunta de novo. É usada em todos os menus e em todas as leituras de código, ano e quantidade.
- **limparTela:** limpa o console usando `system("cls")` no Windows ou `system("clear")` no Linux e no macOS, escolhido pela diretiva `#ifdef _WIN32`.
- **pausar:** exibe “Pressione ENTER para continuar...” e espera o ENTER, descartando o que for digitado antes, para dar tempo de ler as mensagens.
- **obterDataAtual:** obtém a data do relógio do computador com `time` e `localtime` e a escreve no formato DD/MM/AAAA com `strftime`. É usada para preencher as datas de empréstimo e devolução.

7.2 Funções de busca

- **buscarAutorPorCodigo, buscarEditoraPorCodigo, buscarLivroPorCodigo e buscarUsuarioPorCodigo:** percorrem o vetor da entidade com um laço `for` procurando o código informado. Devolvem a posição (índice) do registro no vetor ou -1 se ele não existir. Devolver a posição, e não uma cópia, permite que a função chamadora leia e também altere o registro original.

7.3 Funções de exibição

- **exibirAutores, exibirEditoras, exibirLivros e exibirUsuarios:** imprimem uma tabela alinhada (com `printf` e larguras fixas, como `%-30s`) com todos os registros da entidade, ou uma mensagem avisando que não há registros. São reaproveitadas pelas telas de consulta, alteração, exclusão, cadastro de livro e empréstimo.

- **exibirEmprestimos:** imprime o histórico de empréstimos. Para cada empréstimo, busca o nome do usuário e o título do livro pelos códigos guardados, mostra "(removido)" se o registro não existir mais, exibe "--" quando ainda não há data de devolução e traduz o `status` para "Emprestado" ou "Devolvido".

7.4 CRUD de Autor

- **cadastrarAutor:** verifica se o limite `MAX_AUTORES` foi atingido, gera o código automaticamente (maior código existente + 1), lê nome e nacionalidade, copia a struct para o vetor, incrementa o contador e grava o arquivo.
- **consultarAutores:** limpa a tela, exibe a lista de autores e pausa.
- **alterarAutor:** exibe a lista, pede o código (0 cancela), localiza o autor, mostra os dados atuais e lê o novo nome e a nova nacionalidade, gravando o arquivo em seguida.
- **excluirAutor:** pede o código e, antes de excluir, percorre os livros verificando se algum usa esse autor; se usar, a exclusão é bloqueada. Caso contrário, desloca uma posição para trás todos os registros após o excluído, decrementa o contador e grava o arquivo.

7.5 CRUD de Editora

- **cadastrarEditora, consultarEditoras, alterarEditora e excluirEditora:** seguem a mesma lógica do CRUD de Autor, com os campos nome e cidade. A exclusão é bloqueada se algum livro estiver ligado à editora.

7.6 CRUD de Livro

- **cadastrarLivro:** só permite o cadastro se já existir pelo menos um autor e uma editora. Gera o código, lê o título, mostra a lista de autores e pede o código até que seja válido (0 cancela), faz o mesmo com as editoras, lê o ano e a quantidade de exemplares (que precisa ser maior que zero). A quantidade disponível começa igual à quantidade total.
- **consultarLivros:** exibe a lista de livros e permite ver os detalhes de um deles. Nos detalhes, os códigos de autor e editora são "traduzidos" para os nomes, por meio das funções de busca.
- **alterarLivro:** permite alterar título, ano e quantidade total. Ao mudar o total, a quantidade disponível é ajustada pela diferença entre o novo e o antigo total e limitada para nunca ficar negativa nem maior que o novo total.
- **excluirLivro:** bloqueia a exclusão se houver algum empréstimo em aberto do livro; caso contrário, remove o registro deslocando os demais e grava o arquivo.

7.7 CRUD de Usuário

- **cadastrarUsuario, consultarUsuarios e alterarUsuario:** seguem a mesma lógica das demais entidades, com os campos nome, CPF e telefone.
- **excluirUsuario:** bloqueia a exclusão se o usuário tiver algum empréstimo em aberto; caso contrário, remove o registro.

7.8 Empréstimos e devoluções

- **registrarEmprestimo:** verifica o limite de empréstimos e se existem usuários e livros. Mostra os usuários e pede um código válido; mostra os livros e pede um código válido que tenha pelo menos um exemplar disponível. Gera o número do empréstimo, preenche a data de hoje, deixa a data de devolução vazia, define o status como emprestado, diminui em 1 a quantidade disponível do livro e grava os arquivos de empréstimos e de livros.
- **registrarDevolucao:** lista apenas os empréstimos em aberto, pede o número do empréstimo, recusa números inexistentes ou já devolvidos, preenche a data de devolução com a data de hoje, muda o status para devolvido, aumenta em 1 a quantidade disponível do livro (sem ultrapassar o total) e grava os arquivos.
- **consultarEmprestimos:** exibe o histórico completo de empréstimos, abertos e devolvidos.

7.9 Persistência em arquivos

- **salvarAutores, salvarEditoras, salvarLivros, salvarUsuarios e salvarEmprestimos:** abrem o arquivo da entidade em modo binário de escrita (`"wb"`), gravam primeiro um `int` com a quantidade de registros e depois o vetor inteiro de structs com `fwrite`, e fecham o arquivo. Se o arquivo não puder ser aberto, exibem uma mensagem de erro. São chamadas logo após cada alteração.
- **carregarAutores, carregarEditoras, carregarLivros, carregarUsuarios e carregarEmprestimos:** abrem o arquivo em modo binário de leitura (`"rb"`), leem a quantidade e depois os registros com `fread`. Se o arquivo não existir (primeira execução), começam com zero registros; se a quantidade lida for inválida (negativa ou acima do limite), descartam o conteúdo para evitar leitura fora do vetor.
- **carregarTudo:** chama todas as funções de carregamento de uma vez, no início do programa.

7.10 Menus e função principal

- **menuPrincipal:** exibe as opções de gerenciar Livros, Autores, Editoras, Usuários e Empréstimos, lê a escolha com `lerInteiro` e usa `switch` para chamar o submenu correspondente, repetindo em um laço `do...while` até o usuário escolher 0 (Sair).
- **menuLivros, menuAutores, menuEditoras e menuUsuarios:** submenus com as opções Cadastrar, Consultar, Alterar, Excluir e Voltar, no mesmo padrão `do...while` com `switch`.
- **menuEmprestimos:** submenu com Registrar Empréstimo, Registrar Devolução, Consultar Empréstimos e Voltar.
- **main:** carrega os dados salvos, mostra a tela de boas-vindas com a quantidade de registros de cada entidade, abre o menu principal e, ao sair, encerra o programa.

8 FUNCIONALIDADE EXTRA: EMPRÉSTIMOS E DEVOLUÇÕES

Além das operações CRUD, as orientações da tarefa pedem pelo menos uma funcionalidade adicional que torne a aplicação mais completa. O grupo escolheu implementar o controle de empréstimos e devoluções, por ser a atividade central do dia a dia de uma biblioteca. Esse módulo é o que liga as entidades Usuário e Livro, por meio da struct Emprestimo.

8.1 Como funciona

1. O atendente escolhe o usuário e o livro nas listas exibidas na tela.
2. O sistema verifica se o livro tem pelo menos um exemplar disponível (`quantidadeDisponivel > 0`). Se não tiver, avisa e pede outro livro.
3. O empréstimo é registrado com número automático, a data de hoje e o status “Emprestado”.
4. A quantidade disponível do livro diminui em 1 automaticamente.
5. Na devolução, o atendente escolhe o empréstimo entre os que estão em aberto; o sistema preenche a data de devolução, muda o status para “Devolvido” e aumenta em 1 a quantidade disponível.
6. Os arquivos de empréstimos e de livros são gravados imediatamente, nos dois casos.

8.2 Regras que garantem a consistência

- Não é possível emprestar um livro sem exemplares disponíveis.
- Não é possível devolver duas vezes o mesmo empréstimo.

- A quantidade disponível nunca fica negativa nem maior que o total de exemplares, inclusive quando o total é alterado no cadastro do livro.
- Não é possível excluir um livro ou um usuário que tenha empréstimo em aberto, o que preserva o histórico.
- Não é possível excluir um autor ou uma editora que tenha livros cadastrados, para não deixar livros apontando para registros inexistentes.
- As datas são preenchidas pelo relógio do computador, sem risco de erro de digitação.

O funcionamento desse módulo aparece nas Figuras 16 a 21 da seção 10.

9 VALIDAÇÃO DE DADOS E TRATAMENTO DE ERROS

O sistema trata de forma simples os erros mais comuns de digitação e de uso, sem travar e sem aceitar dados que deixariam o sistema inconsistente. A Tabela 10 reúne as validações implementadas.

Tabela 10 – Validações e tratamento de erros

Situação	Tratamento no sistema
Texto ou valor vazio em campo numérico (menus, códigos, ano, quantidade)	`lerInteiro` recusa e pede novamente: “Valor invalido. Digite somente numeros.”
Número seguido de letras (ex.: “12a”)	Recusado pela mesma verificação do `strtol`.
Opção inexistente no menu	Mensagem “Opcao invalida!” e retorno ao menu.
Texto maior que o campo da struct	`lerLinha` guarda só o que cabe e descarta o restante da linha.
Código de autor, editora, usuário ou livro inexistente	Mensagem de “não encontrado”; no cadastro de livro e no empréstimo, a pergunta é repetida.
Quantidade de exemplares zero ou negativa	Recusada: “A quantidade deve ser maior que zero.”
Cadastro de livro sem autor ou editora cadastrados	Bloqueado com mensagem explicando o que cadastrar antes.
Empréstimo de livro sem exemplar disponível	“Nao ha exemplares disponiveis deste livro no momento.”
Devolução de empréstimo inexistente ou já devolvido	Recusada com mensagem específica.
Exclusão de registro ainda em uso	Bloqueada (ver seção 8.2).
Limite do vetor atingido (`MAX_*`)	Novo cadastro bloqueado com mensagem de limite máximo.
Falha ao abrir arquivo para gravar	Mensagem de erro ao salvar.

Arquivo inexistente ou corrompido ao carregar	Sistema começa com zero registros em vez de ler dados inválidos.
Operação iniciada por engano	Digitar 0 no código cancela alteração, exclusão, cadastro de livro, empréstimo e devolução.

Fonte: elaborado pelos autores (2026).

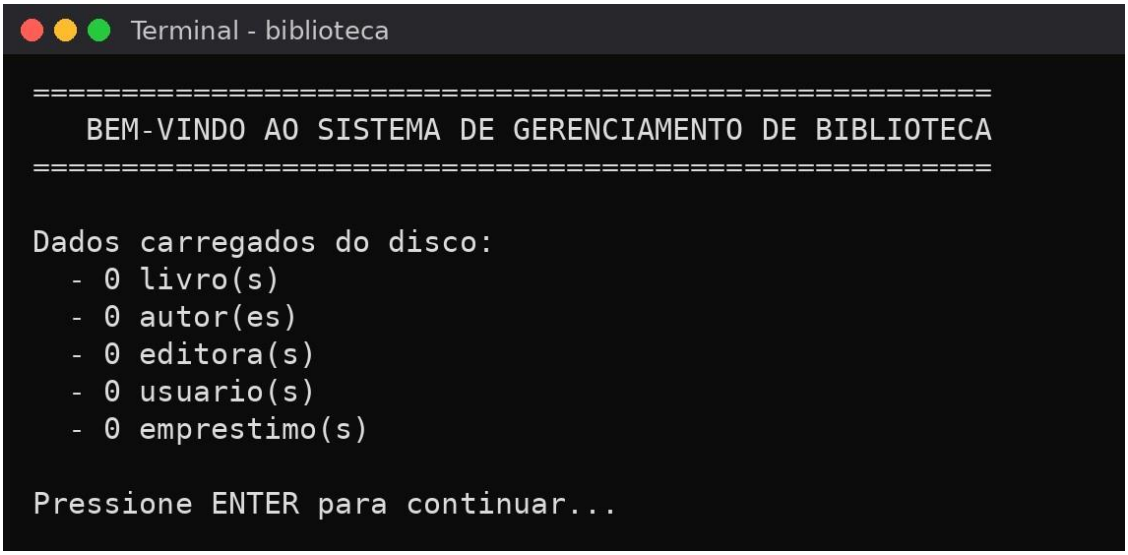
10 DESCRIÇÃO DO PROJETO E INTERFACE DO USUÁRIO

O sistema funciona inteiramente no console (Prompt de Comando no Windows ou Terminal no Linux e no macOS). A navegação é feita por menus numerados: o usuário digita o número da opção e pressiona ENTER. Os textos do programa foram escritos sem acentuação de propósito, para evitar que os acentos apareçam como símbolos estranhos no Prompt de Comando do Windows.

As telas a seguir foram capturadas a partir de uma execução real do programa, seguindo um roteiro de teste com dados fictícios: dois autores (Machado de Assis e George Orwell), duas editoras, dois livros (Dom Casmurro, com 3 exemplares, e 1984, com 1 exemplar, depois alterado para 2) e três usuários, sendo um deles excluído durante o teste. Os CPFs e telefones exibidos são fictícios.

10.1 Início do sistema e menu principal

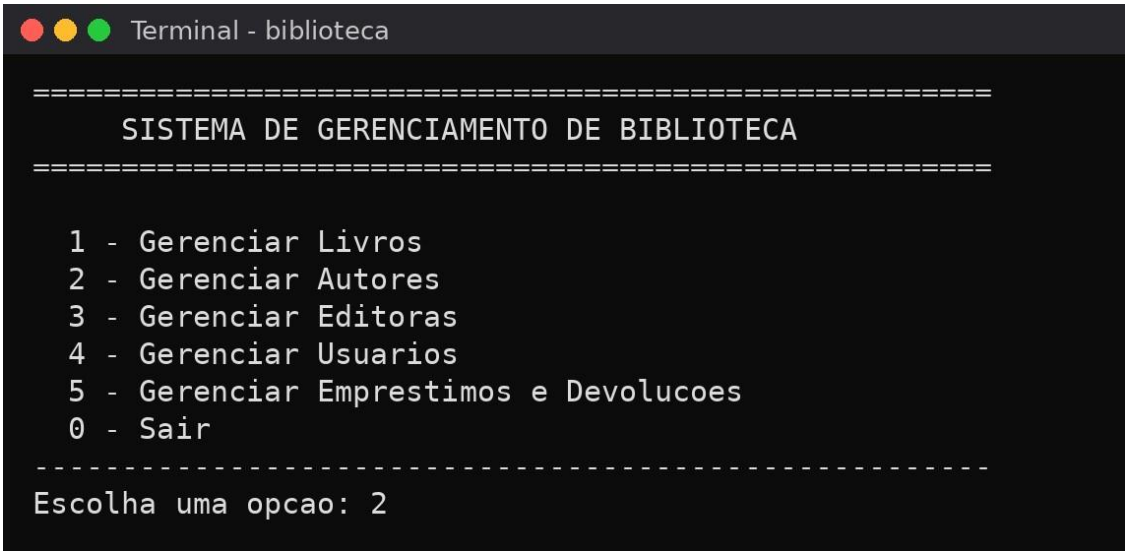
Figura 1 – Tela de boas-vindas



Fonte: elaborado pelos autores (2026).

Ao abrir o programa, os dados salvos nos arquivos são carregados e a tela de boas-vindas mostra quantos registros de cada entidade foram encontrados. Na primeira execução, todos os contadores aparecem zerados, como na Figura 1.

Figura 2 – Menu principal

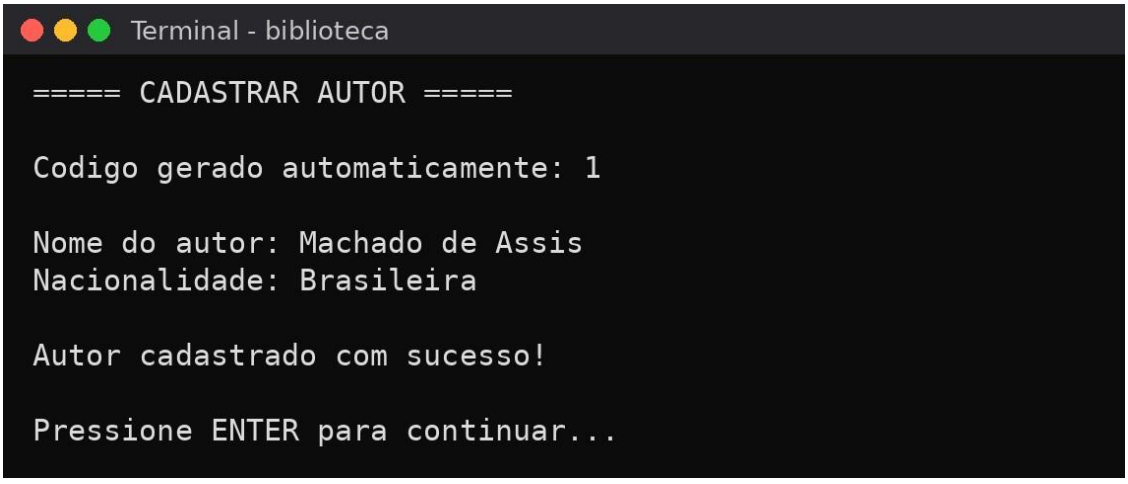


Fonte: elaborado pelos autores (2026).

O menu principal (Figura 2) dá acesso aos cinco módulos do sistema. Cada opção abre um submenu próprio; a opção 0 encerra o programa.

10.2 Inclusão de registros

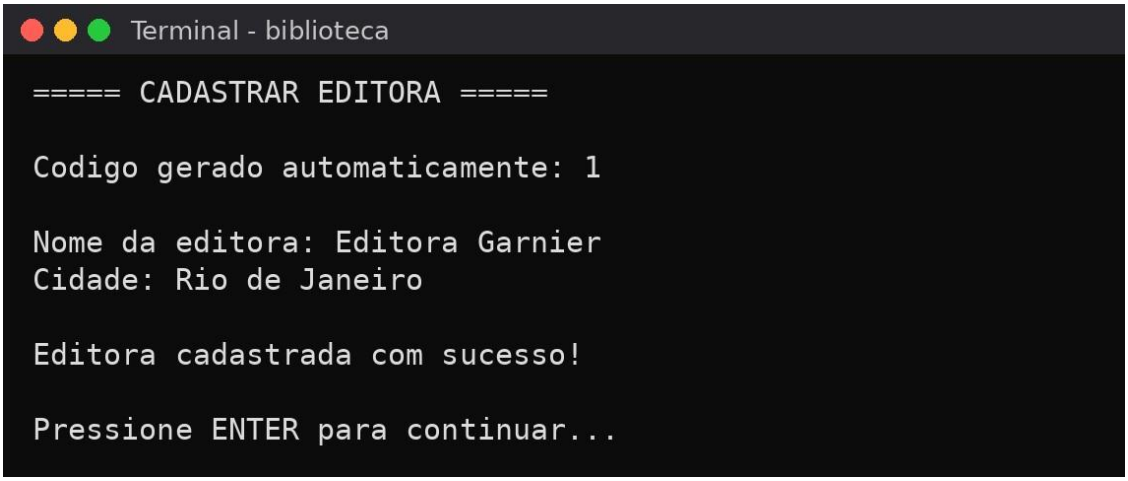
Figura 3 – Cadastro de autor



Fonte: elaborado pelos autores (2026).

No cadastro de autor (Figura 3), o código é gerado automaticamente e o usuário informa apenas o nome e a nacionalidade. Ao final, o registro é gravado no arquivo e a mensagem de sucesso é exibida.

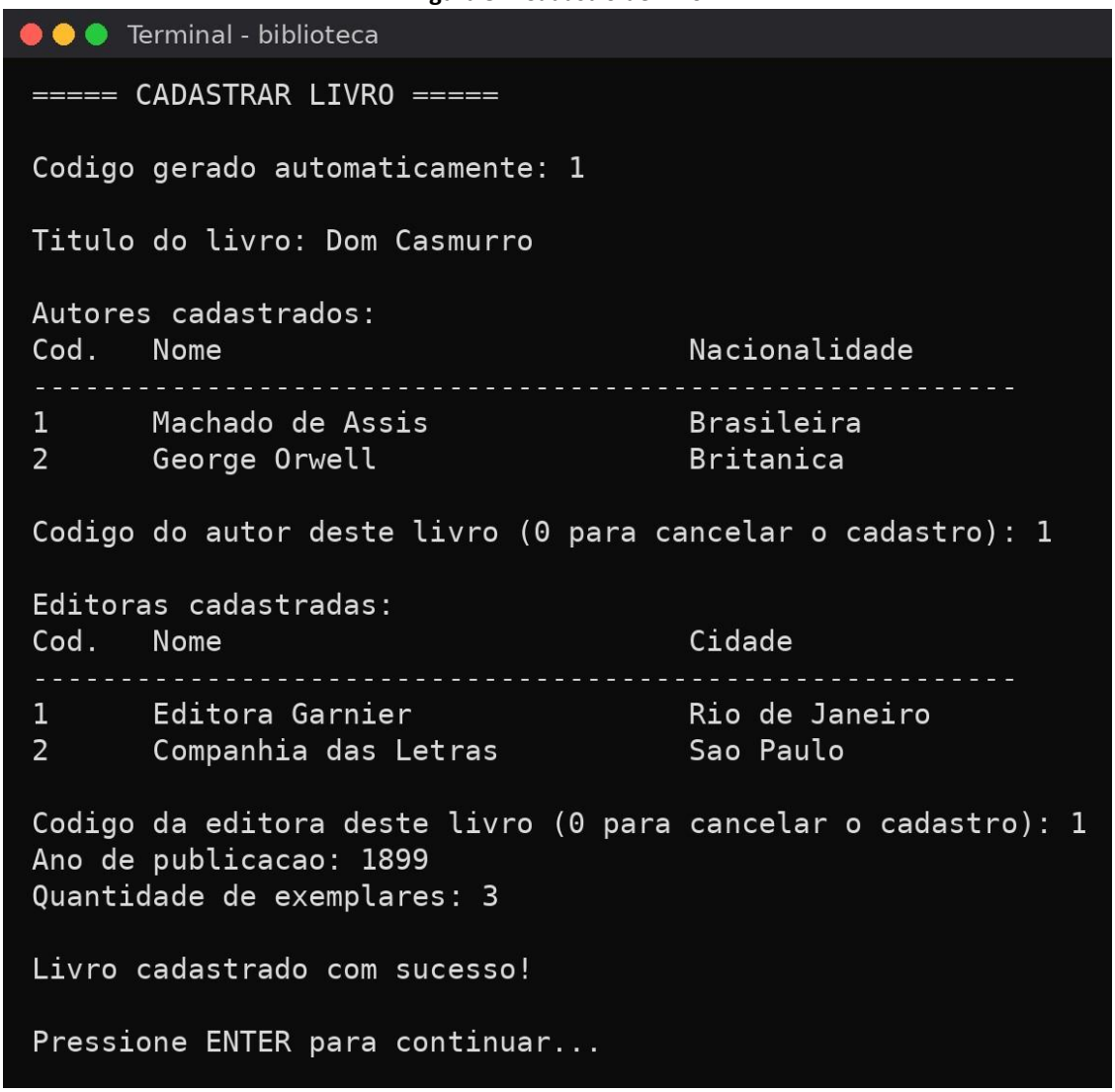
Figura 4 – Cadastro de editora



Fonte: elaborado pelos autores (2026).

O cadastro de editora (Figura 4) segue o mesmo padrão, com os campos nome e cidade.

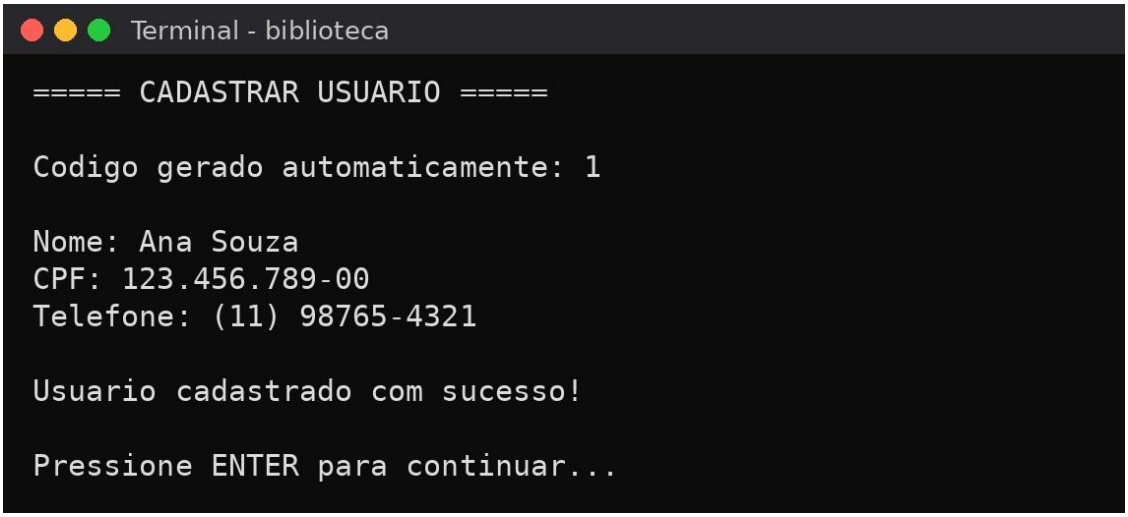
Figura 5 – Cadastro de livro



Fonte: elaborado pelos autores (2026).

No cadastro de livro (Figura 5), depois do título, o sistema mostra as listas de autores e editoras para que o usuário escolha os códigos, em vez de precisar decorálos. Em seguida são informados o ano de publicação e a quantidade de exemplares.

Figura 6 – Cadastro de usuário

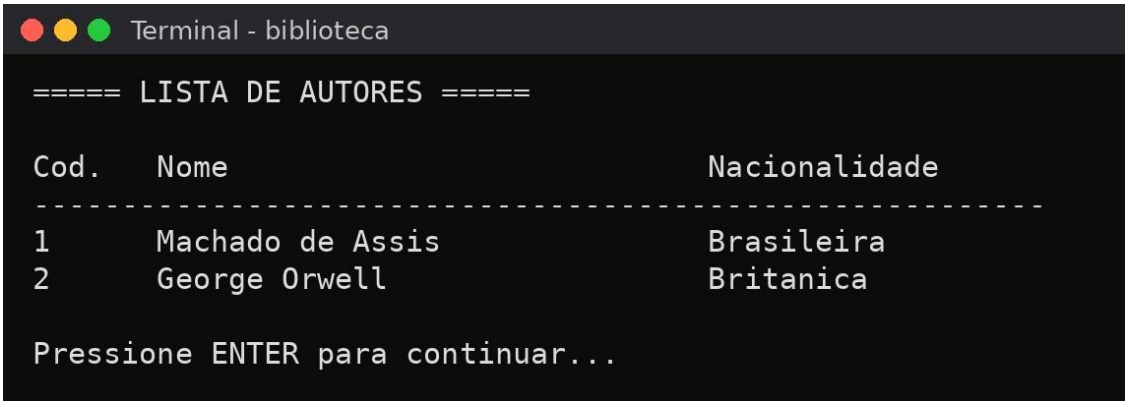


Fonte: elaborado pelos autores (2026).

No cadastro de usuário (Figura 6) são informados nome, CPF e telefone.

10.3 Listagem de registros

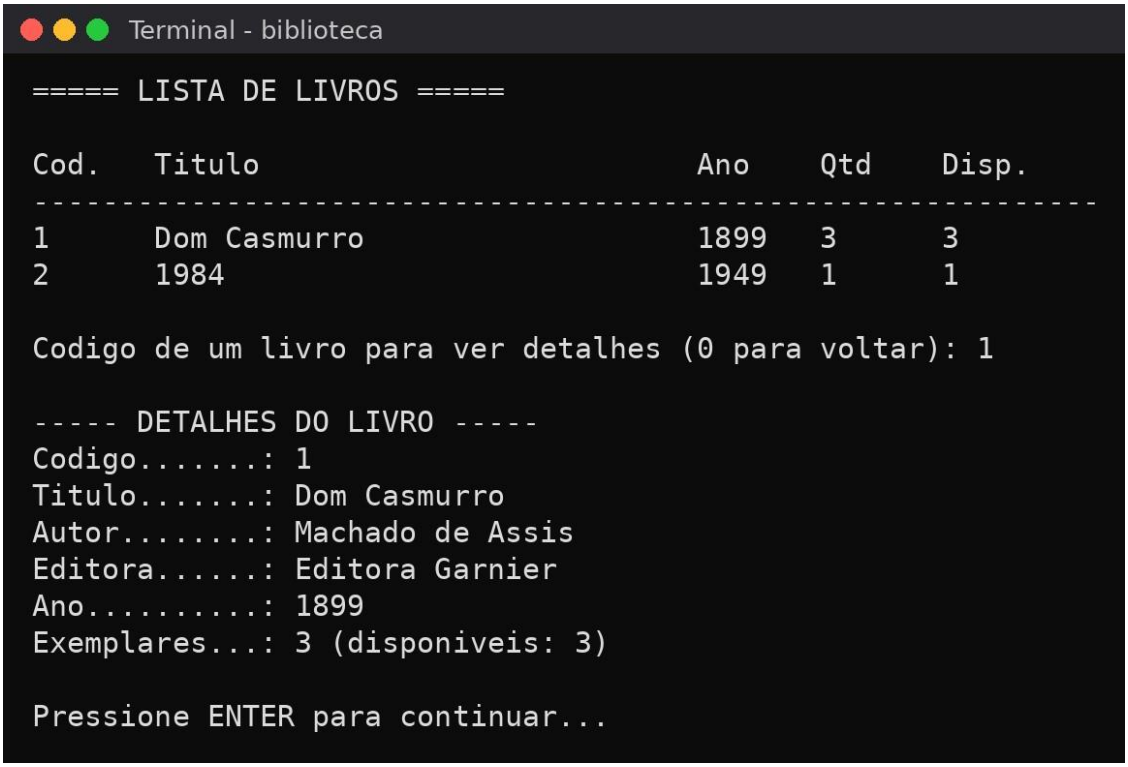
Figura 7 – Listagem de autores



Fonte: elaborado pelos autores (2026).

A consulta de autores (Figura 7) exibe todos os registros em uma tabela alinhada.

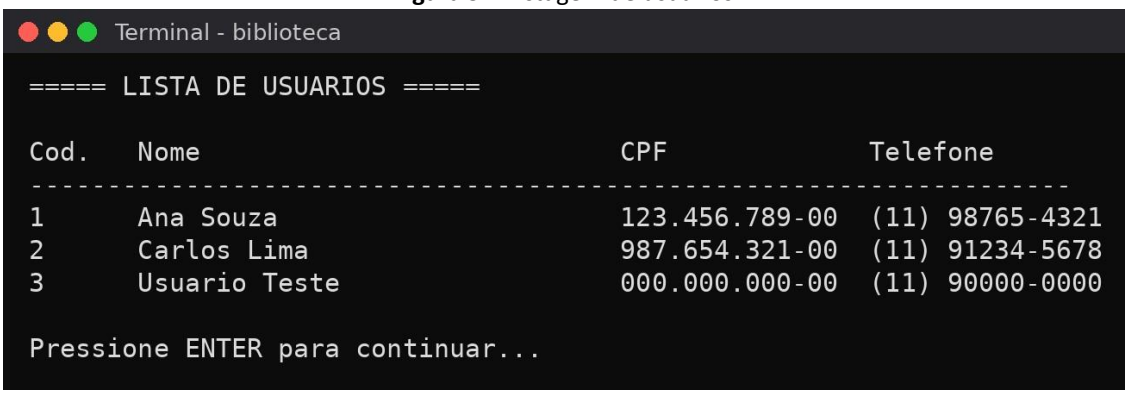
Figura 8 – Listagem de livros e detalhes de um livro



Fonte: elaborado pelos autores (2026).

A consulta de livros (Figura 8) mostra, além do título e do ano, a quantidade total e a disponível de cada livro. Ao informar um código, o sistema exibe os detalhes do livro, com os nomes do autor e da editora obtidos pelos códigos guardados na struct.

Figura 9 – Listagem de usuários



Fonte: elaborado pelos autores (2026).

A Figura 9 mostra a lista de usuários cadastrados, com CPF e telefone.

10.4 Edição de registros

Figura 10 – Alteração de livro

```
Terminal - biblioteca

===== ALTERAR LIVRO =====

Cod.   Titulo                          Ano    Qtd    Disp.
-----------------------------------------------------------
1      Dom Casmurro                    1899   3      3
2      1984                            1949   1      1

Codigo do livro a alterar (0 para cancelar): 2

Titulo atual: 1984
Novo titulo: 1984 (Ed. Especial)

Ano atual: 1949
Novo ano: 1949

Quantidade atual: 1 (disponiveis: 1)
Nova quantidade total de exemplares: 2

Livro atualizado com sucesso!

Pressione ENTER para continuar...
```

Fonte: elaborado pelos autores (2026).

Na alteração de livro (Figura 10), o sistema mostra cada valor atual antes de pedir o novo. No exemplo, o título do livro 1984 foi alterado e a quantidade total passou de 1 para 2 exemplares; a quantidade disponível foi ajustada automaticamente.

Figura 11 – Alteração de usuário

```
Terminal - biblioteca

===== ALTERAR USUARIO =====

Cod.   Nome                    CPF            Telefone
-----------------------------------------------------------------
1      Ana Souza               123.456.789-00  (11) 98765-4321
2      Carlos Lima             987.654.321-00  (11) 91234-5678
3      Usuario Teste           000.000.000-00  (11) 90000-0000

Codigo do usuario a alterar (0 para cancelar): 2

Dados atuais -> Nome: Carlos Lima | CPF: 987.654.321-00 | Telefone: (11) 91234-5678
Novo nome: Carlos Lima Pereira
Novo CPF: 987.654.321-00
Novo telefone: (11) 95555-1234

Usuario atualizado com sucesso!

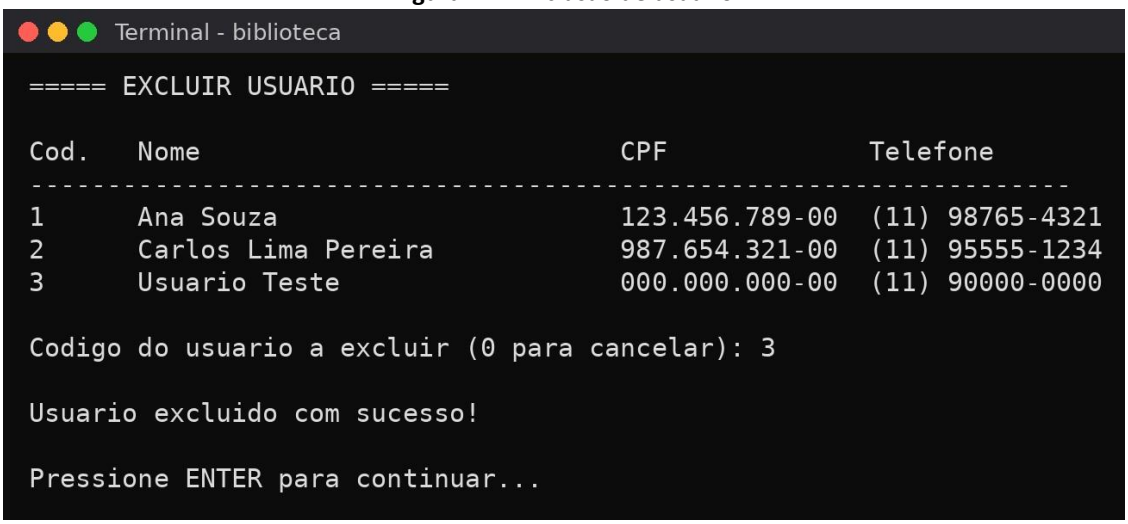
Pressione ENTER para continuar...
```

Fonte: elaborado pelos autores (2026).

Na alteração de usuário (Figura 11), os dados atuais aparecem em uma linha e o usuário digita o novo nome, CPF e telefone.

10.5 Exclusão de registros

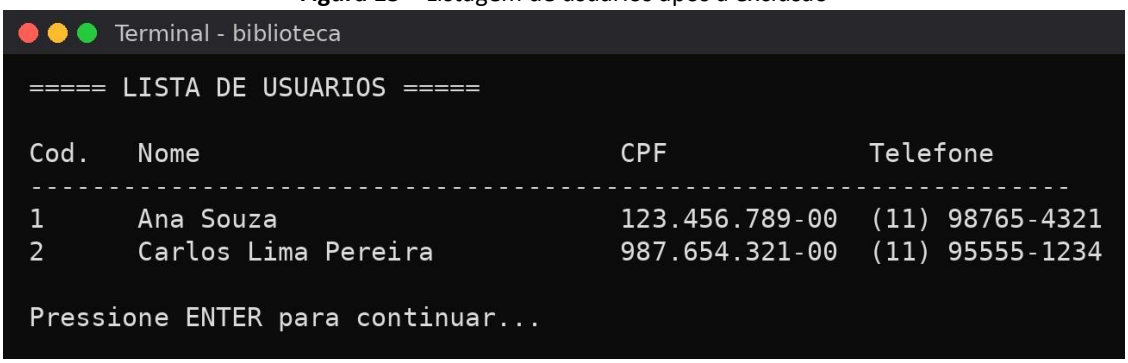
Figura 12 – Exclusão de usuário



Fonte: elaborado pelos autores (2026).

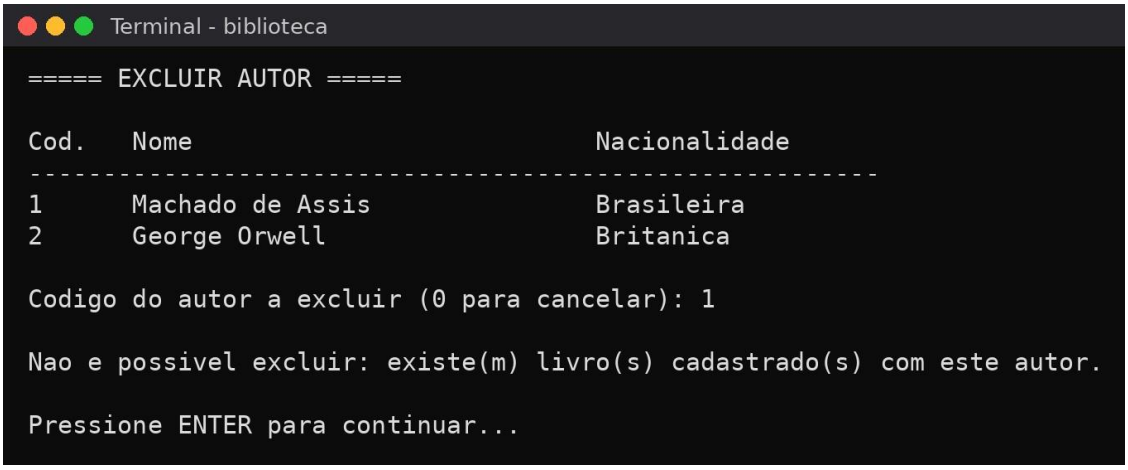
Na exclusão (Figura 12), o sistema mostra a lista, pede o código e remove o registro. Na Figura 13, a nova listagem confirma que o usuário 3 foi excluído.

Figura 13 – Listagem de usuários após a exclusão



Fonte: elaborado pelos autores (2026). **Figura 14**

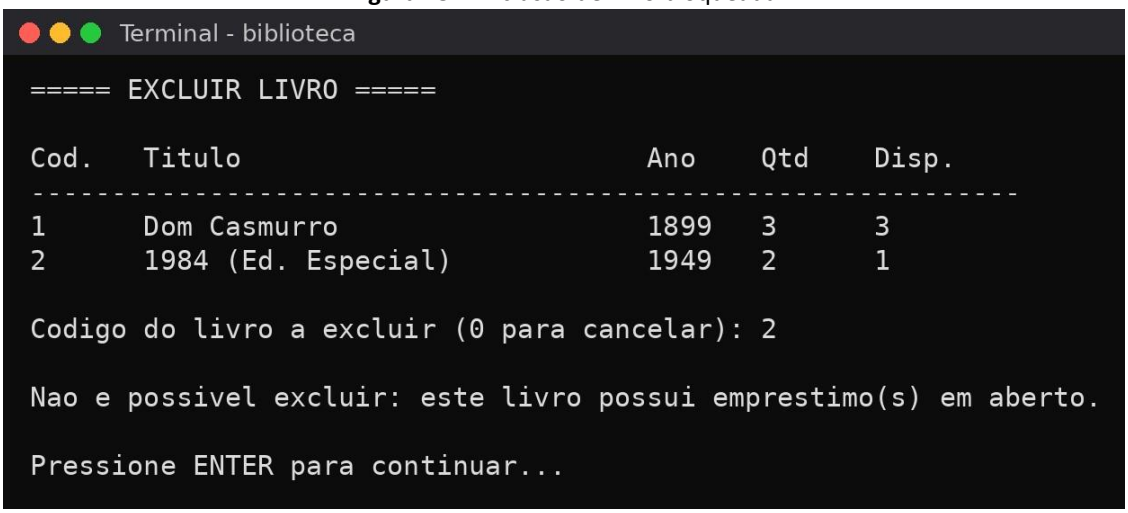
– Exclusão de autor bloqueada



Fonte: elaborado pelos autores (2026).

Quando a exclusão deixaria dados inconsistentes, ela é bloqueada. Na Figura 14, o autor Machado de Assis não pode ser excluído porque o livro Dom Casmurro está ligado a ele.

Figura 15 – Exclusão de livro bloqueada

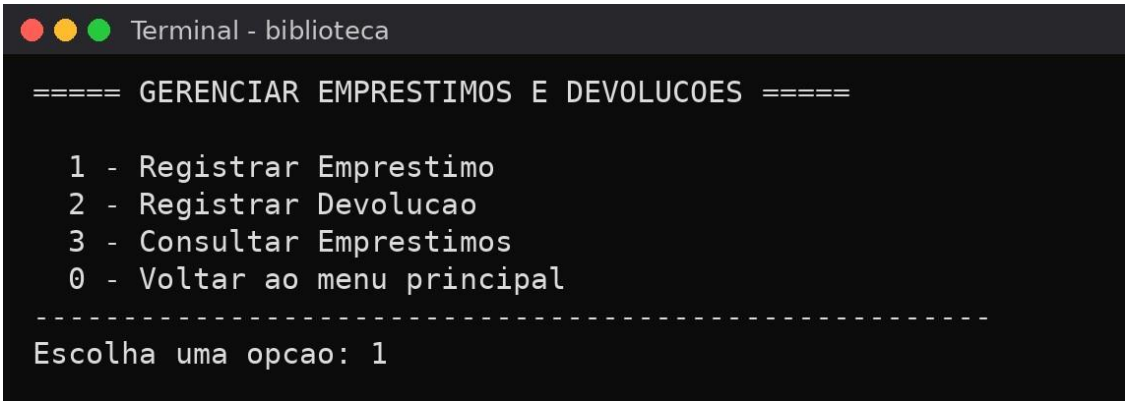


Fonte: elaborado pelos autores (2026).

Na Figura 15, o livro 1984 não pode ser excluído porque ainda possui um empréstimo em aberto.

10.6 Funcionalidade extra em ação: empréstimos e devoluções

Figura 16 – Menu de empréstimos e devoluções



Fonte: elaborado pelos autores (2026).

O submenu de empréstimos (Figura 16) reúne as três operações do módulo extra.

Figura 17 – Registro de empréstimo



Fonte: elaborado pelos autores (2026).

No registro de empréstimo (Figura 17), o usuário escolhe o leitor e o livro. O empréstimo recebe número e data automáticos. Note que o livro 1984 aparecia com 2 exemplares disponíveis antes deste empréstimo.

Figura 18 – Empréstimo recusado por falta de exemplares

```
Terminal - biblioteca
===== REGISTRAR EMPRESTIMO =====

Usuarios cadastrados:
Cod.   Nome                          CPF             Telefone
-----------------------------------------------------------------
1      Ana Souza                     123.456.789-00  (11) 98765-4321
2      Carlos Lima Pereira           987.654.321-00  (11) 95555-1234

Codigo do usuario (0 para cancelar): 1

Livros cadastrados:
Cod.   Titulo                        Ano    Qtd    Disp.
-------------------------------------------------------------
1      Dom Casmurro                  1899   3      3
2      1984 (Ed. Especial)           1949   2      0

Codigo do livro (0 para cancelar): 2
  >> Nao ha exemplares disponiveis deste livro no momento.

Codigo do livro (0 para cancelar): 0

Operacao cancelada.

Pressione ENTER para continuar...
```

Fonte: elaborado pelos autores (2026).

Depois de dois empréstimos, o livro 1984 ficou com 0 exemplares disponíveis. Ao tentar um terceiro empréstimo (Figura 18), o sistema recusa e pede outro livro; digitando 0, a operação é cancelada.

Figura 19 – Consulta de empréstimos em aberto

```
Terminal - biblioteca
===== LISTA DE EMPRESTIMOS =====

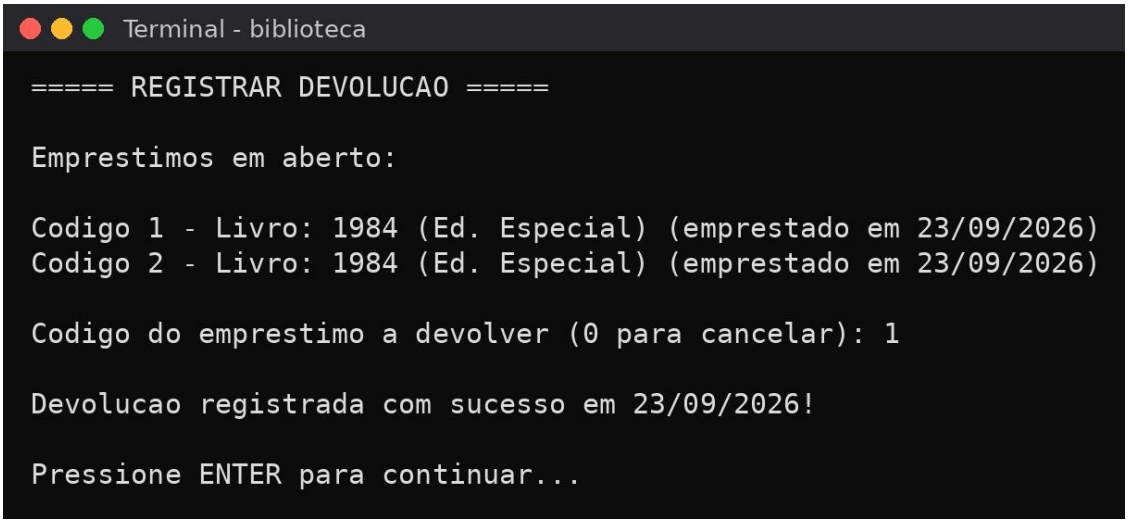
Cod.   Usuario           Livro              Emprestimo   Devolucao    Status
-------------------------------------------------------------------------------
1      Ana Souza         1984 (Ed. Especial)  23/09/2026   --           Emprestado
2      Carlos Lima Pereira  1984 (Ed. Especial)  23/09/2026   --           Emprestado

Pressione ENTER para continuar...
```

Fonte: elaborado pelos autores (2026).

A consulta de empréstimos (Figura 19) mostra o histórico, com o nome do usuário e o título do livro obtidos pelos códigos, a data de retirada e a situação. Os empréstimos ainda não devolvidos aparecem com “--” na coluna de devolução.

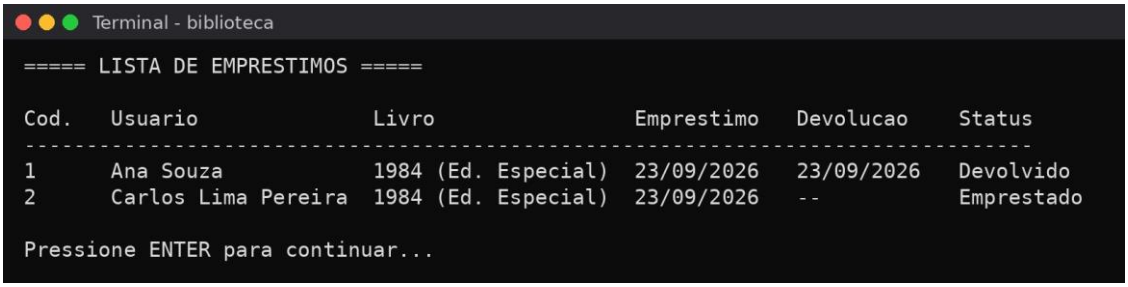
Figura 20 – Registro de devolução



Fonte: elaborado pelos autores (2026).

No registro de devolução (Figura 20), o sistema lista apenas os empréstimos em aberto. Ao escolher um deles, a data de devolução é preenchida automaticamente e um exemplar volta a ficar disponível.

Figura 21 – Consulta de empréstimos após a devolução

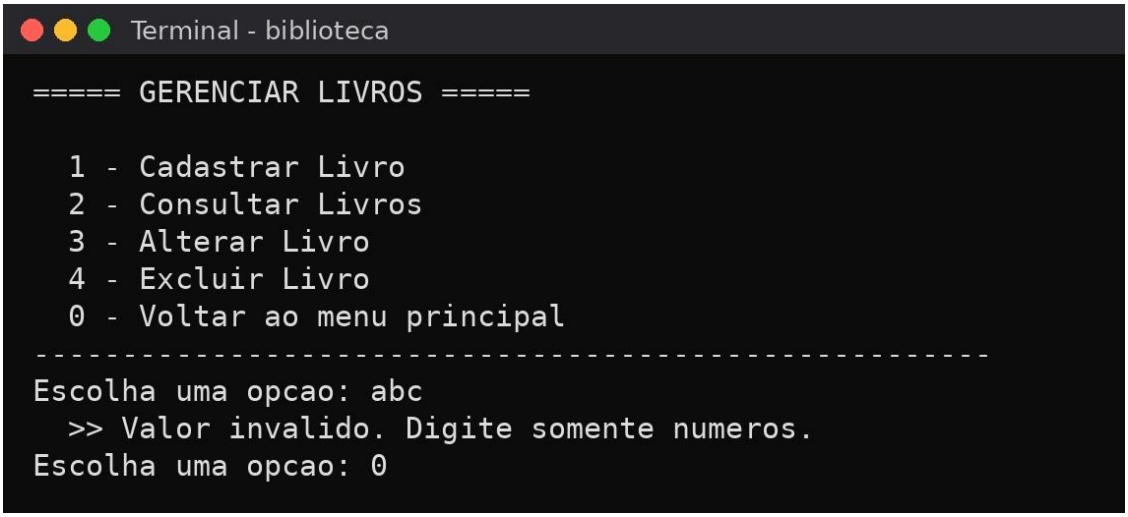


Fonte: elaborado pelos autores (2026).

Na nova consulta (Figura 21), o empréstimo 1 aparece como “Devolvido”, com a data de devolução, enquanto o empréstimo 2 continua em aberto.

10.7 Tratamento de erros e encerramento

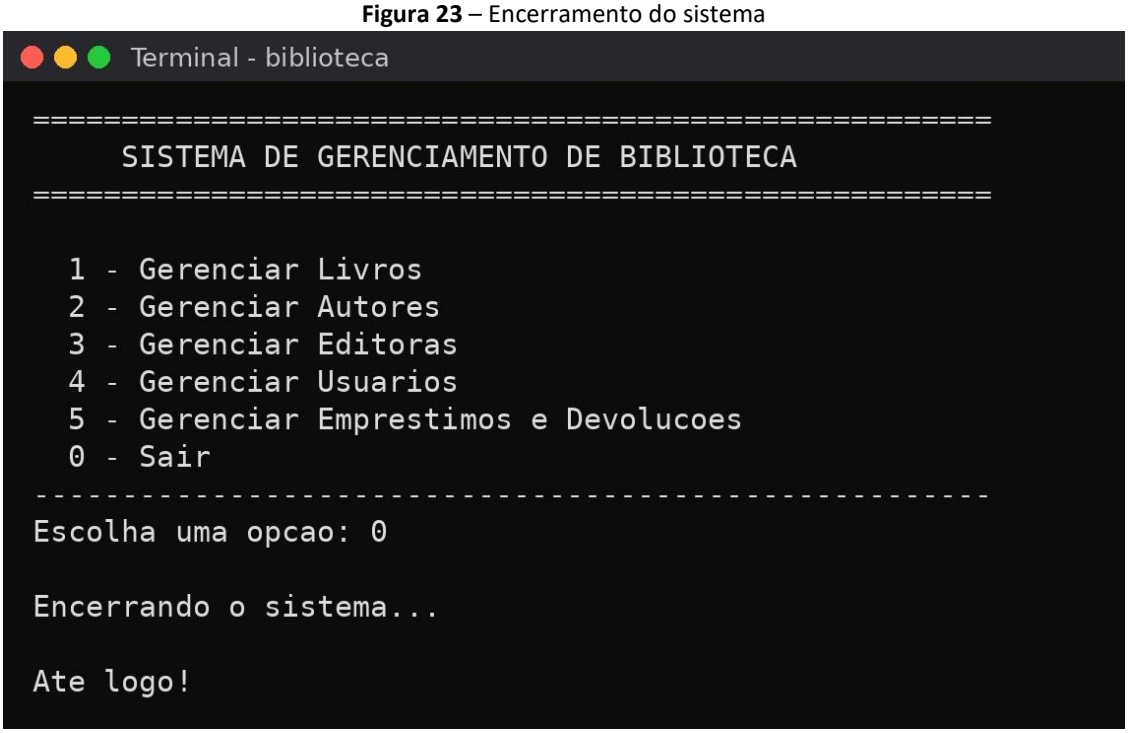
Figura 22 – Tratamento de entrada inválida no menu



Fonte: elaborado pelos autores (2026).

Na Figura 22, o usuário digitou “abc” em vez de um número. O sistema avisa que o valor é inválido e repete a pergunta, sem travar.

Figura 23 – Encerramento do sistema



Fonte: elaborado pelos autores (2026).

Ao escolher 0 no menu principal (Figura 23), o programa é encerrado. Como os dados são gravados logo após cada alteração, nenhuma informação se perde.

11 CONSIDERAÇÕES FINAIS

O projeto atingiu o objetivo proposto: foi desenvolvido um sistema de gerenciamento de biblioteca funcional em linguagem C, com operações CRUD completas para livros, autores,

editoras e usuários e com o controle de empréstimos e devoluções como funcionalidade extra. As cinco estruturas definidas na entrega parcial foram mantidas sem alterações e se mostraram suficientes para representar as informações e os relacionamentos necessários.

O desenvolvimento permitiu ao grupo aplicar de forma integrada os conceitos estudados na disciplina. As structs organizaram os dados; os vetores e contadores guardaram os registros em memória; as funções dividiram o problema em partes pequenas e reutilizáveis; as estruturas de decisão e repetição implementaram os menus, as buscas e as validações; e a manipulação de arquivos binários garantiu que os dados permaneçam salvos entre uma execução e outra.

Como limitações, os vetores têm tamanho fixo, a busca é feita apenas por código, e os campos de CPF e telefone não têm verificação de formato. Como melhorias futuras, o grupo identifica: validação do CPF e do telefone, impedimento de CPF duplicado, busca de livros por título ou autor, relatório de livros mais emprestados, controle de prazo de devolução com aviso de atraso e uso de alocação dinâmica de memória para eliminar os limites fixos.

REFERÊNCIAS

BRASIL. Lei nº 12.244, de 24 de maio de 2010. Dispõe sobre a universalização das bibliotecas nas instituições de ensino do País. **Diário Oficial da União**, Brasília, DF, 25 maio 2010.

DEITEL, Paul; DEITEL, Harvey. **C: como programar**. 6. ed. São Paulo: Pearson Prentice Hall, 2011.

FEOFILOFF, Paulo. **Algoritmos em linguagem C**. Rio de Janeiro: Elsevier, 2009.

FORBELLONE, André Luiz Villar; EBERSPÄCHER, Henri Frederico. **Lógica de programação**: a construção de algoritmos e estruturas de dados. 3. ed. São Paulo: Pearson Prentice Hall, 2005.

MOKARZEL, Fábio; SOMA, Nei. **Introdução à ciência da computação**. Rio de Janeiro: Elsevier, 2008.

SANTOS, Júlio Cesar Grilo dos et al.. **Gerenciador de biblioteca**. GitHub, 2026. Disponível em: https://github.com/JulioCesarGrilo/Gerenciador-de-biblioteca. Acesso em: 23 set. 2026.

APÊNDICE A – CÓDIGO DO PROJETO

Código-fonte completo do arquivo `projetocrud.c`, também disponível no repositório do projeto no GitHub.

```
/* ============================================================================
*                   SISTEMA DE GERENCIAMENTO DE BIBLIOTECA - LINGUAGEM C
*                   Universidade Sao Francisco - Linguagem de Programacao
*                    *  O sistema permite cadastrar, consultar, alterar e
                    excluir (CRUD):
*                   - Autores
*                   - Editoras
*                   - Livros
*                   - Usuarios
*                   E tambem controlar Emprestimos e Devolucoes de livros,
                    atualizando *  automaticamente a quantidade de exemplares
                    disponiveis.
 *
*                   Todos os dados sao salvos em arquivos binarios na mesma
                    pasta do *  programa, entao nada se perde quando o programa
                    e fechado.
 *
*                   COMO COMPILAR:   gcc projetocrud.c -o biblioteca
*                   COMO EXECUTAR:   ./biblioteca        (Linux/Mac)
*                   biblioteca.exe      (Windows)
*                    *  OBS: os textos do programa foram escritos sem acentuacao
                    de proposito.
*                   Isso evita o problema classico de acentos aparecendo como
                    simbolos
*                   estranhos no Prompt de Comando do Windows quando o
                    compilador nao usa *  UTF-8 por padrao (um problema muito
                    comum no Dev-C++/Code::Blocks).
*                   ============================================================
                    ============== */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

/* ----------------------------------------------------------------------
*                   CONSTANTES DO SISTEMA
*                   MAX_... define o tamanho de cada vetor, ou seja, quantos
                    registros
*                   cada entidade pode guardar ao mesmo tempo. ARQ_... e o nome
                    do *  arquivo em disco usado para guardar cada tipo de dado.
*                   ------------------------------------------------------------
                    ---------- */
#define MAX_LIVROS       200
#define MAX_AUTORES      100
#define MAX_EDITORAS     100
#define MAX_USUARIOS     200
#define MAX_EMPRESTIMOS  500

#define ARQ_LIVROS       "livros.dat"
#define ARQ_AUTORES      "autores.dat"
```

```
#define ARQ_EDITORAS     "editoras.dat"
#define ARQ_USUARIOS     "usuarios.dat"
#define ARQ_EMPRESTIMOS  "emprestimos.dat"

#define STATUS_EMPRESTADO 0   /* emprestimo em aberto, livro ainda com o usuario */
#define STATUS_DEVOLVIDO  1   /* emprestimo encerrado, livro ja devolvido        */ /* ---
------------------------------------------------------------------
*  ESTRUTURAS (structs) *  Cada struct agrupa as informacoes de uma "entidade"
   do sistema.
*  Mantidas exatamente como definidas no planejamento do projeto.
*  ----------------------------------------------------------------------
   */struct Livro {     int codigo;                 /* identificador unico do
   livro            */     char titulo[100];            /* titulo do livro
   */     int codigoAutor;             /* liga o livro a um registro de Autor
   */     int codigoEditora;           /* liga o livro a um registro de
   Editora   */     int ano;                     /* ano de publicacao
   */     int quantidade;              /* total de exemplares que a biblioteca
   tem */     int quantidadeDisponivel;    /* quantos exemplares NAO estao
   emprestados */ };
 struct Autor {     int
codigo;     char nome[100];
char nacionalidade[50]; };
 struct Editora {
int codigo;     char
nome[100];     char
cidade[50]; };
 struct Usuario {
int codigo;     char
nome[100];     char
cpf[15];     char
telefone[20]; };
 struct Emprestimo {     int codigo;     int codigoUsuario;           /* liga o
emprestimo a um Usuario */     int codigoLivro;             /* liga o emprestimo
a um Livro    */       char dataEmprestimo[11];       /* formato DD/MM/AAAA (10
caracteres + '\0') */     char dataDevolucao[11];      /* fica vazia ("") ate o
livro ser devolvido */     int status;                  /* STATUS_EMPRESTADO ou
STATUS_DEVOLVIDO     */ };

/* ----------------------------------------------------------------------
*  VARIAVEIS GLOBAIS
*  Os vetores guardam todos os registros enquanto o programa roda na
*  memoria RAM. As variaveis "total..." dizem quantas posicoes de cada *  vetor
   estao realmente sendo usadas (o resto do vetor fica "vazio").
*  Sao globais porque quase toda funcao do sistema precisa ler ou *  alterar
   esses dados - assim evitamos passar 10 parametros toda hora.
*  ----------------------------------------------------------------------
   */struct Livro livros[MAX_LIVROS]; int totalLivros = 0;  struct Autor
   autores[MAX_AUTORES]; int totalAutores = 0;  struct Editora
   editoras[MAX_EDITORAS];
int totalEditoras = 0;  struct Usuario
usuarios[MAX_USUARIOS]; int totalUsuarios = 0;
struct Emprestimo emprestimos[MAX_EMPRESTIMOS];
int totalEmprestimos = 0;
/* ----------------------------------------------------------------------
*  PROTOTIPOS DAS FUNCOES
*  Avisam o compilador que essas funcoes existem antes de serem usadas,
*  permitindo que uma funcao chame outra que so foi escrita mais abaixo *  no
   arquivo.
```

```
*  ------------------------------------------------------------------- */

/* Utilitarios de entrada e tela */ void
lerLinha(char *destino, int tamanho); int
lerInteiro(const char *mensagem); void
limparTela(void); void pausar(void);
void obterDataAtual(char *destino);
/* Busca (retornam a POSICAO no vetor, ou -1 se nao encontrar) */
int buscarAutorPorCodigo(int codigo); int
buscarEditoraPorCodigo(int codigo); int buscarLivroPorCodigo(int
codigo); int buscarUsuarioPorCodigo(int codigo);
/* Exibicao das tabelas na tela */
void exibirAutores(void); void
exibirEditoras(void); void
exibirLivros(void); void
exibirUsuarios(void); void
exibirEmprestimos(void);
/* CRUD - Autor */ void
cadastrarAutor(void); void
consultarAutores(void); void
alterarAutor(void); void
excluirAutor(void);
/* CRUD - Editora */ void
cadastrarEditora(void); void
consultarEditoras(void); void
alterarEditora(void); void
excluirEditora(void);
/* CRUD - Livro */ void
cadastrarLivro(void); void
consultarLivros(void); void
alterarLivro(void); void
excluirLivro(void);
/* CRUD - Usuario */ void
cadastrarUsuario(void); void
consultarUsuarios(void); void
alterarUsuario(void); void
excluirUsuario(void);

/* Emprestimos e devolucoes */
void registrarEmprestimo(void);
void registrarDevolucao(void);
void consultarEmprestimos(void);
/* Persistencia (gravar/ler os arquivos em disco) */
void salvarAutores(void); void
carregarAutores(void); void salvarEditoras(void);
void carregarEditoras(void); void
salvarLivros(void); void carregarLivros(void); void
salvarUsuarios(void); void carregarUsuarios(void);
void salvarEmprestimos(void); void
carregarEmprestimos(void); void carregarTudo(void);
/* Menus */ void
menuPrincipal(void); void
menuLivros(void); void
menuAutores(void); void
menuEditoras(void); void
menuUsuarios(void); void
menuEmprestimos(void);
```

```
/* ========================================================================
*  FUNCAO PRINCIPAL (main)
*  E o ponto de partida do programa: carrega os dados salvos, mostra uma
*  mensagem de boas-vindas e entra no menu principal. Quando o menu *  principal
   termina (usuario escolheu Sair), o programa se encerra.
*  ========================================================================
   */int main(void) {     carregarTudo();      limparTela();
   printf("=========================================================\n");
   printf("   BEM-VINDO AO SISTEMA DE GERENCIAMENTO DE BIBLIOTECA\n");
   printf("=========================================================\n\n");
   printf("Dados carregados do disco:\n");     printf("  - %d livro(s)\n",
   totalLivros);     printf("  - %d autor(es)\n", totalAutores);     printf("  -
   %d editora(s)\n", totalEditoras);     printf("  - %d usuario(s)\n",
   totalUsuarios);     printf("  - %d emprestimo(s)\n", totalEmprestimos);
   pausar();      menuPrincipal();      printf("\nAte logo!\n");     return 0; }


/* ========================================================================
*  FUNCOES UTILITARIAS
*  ======================================================================== */

/* ------------------------------------------------------------------------
*  lerLinha *  Le uma linha de texto digitada pelo usuario e guarda em
   "destino".
*  Usamos fgets (em vez de scanf) porque fgets le a linha inteira de
*  uma vez, incluindo espacos, e evita o classico bug em C de misturar *
   leitura de numero com leitura de texto e "sobrar" um Enter no buffer.
*  ------------------------------------------------------------------------ */void
   lerLinha(char *destino, int tamanho) {     if (fgets(destino, tamanho, stdin)
   == NULL) {         /* Entrada terminou de forma inesperada (fim de arquivo,
   por            exemplo). Para o programa nao "quebrar", tratamos como uma
   linha vazia. */         destino[0] = '\0';         return;     }
     int len = strlen(destino);      if (len > 0 && destino[len
- 1] == '\n') {         /* Remove o '\n' que o fgets deixa no
final da string */         destino[len - 1] = '\0';     } else
if (len == tamanho - 1) {
        /* O texto digitado era maior que o buffer: ainda sobrou parte
da linha esperando no teclado. Descartamos essa sobra para
que ela nao seja lida sem querer na proxima pergunta. */         int
c;         while ((c = getchar()) != '\n' && c != EOF) {
/* apenas descarta os caracteres */
        }
    }
}

/* ------------------------------------------------------------------------
*  lerInteiro
*  Mostra uma mensagem, le uma linha de texto e converte para numero
*  inteiro usando strtol. Se o usuario digitar algo que nao seja um
*  numero valido (letras, vazio, etc.), o programa avisa e pede de *  novo, em
   vez de travar ou aceitar lixo.
*  ------------------------------------------------------------------------ */int
   lerInteiro(const char *mensagem) {     char buffer[50];     char *sobra;
   long valor;      while (1) {         printf("%s", mensagem);
   lerLinha(buffer, sizeof(buffer));          valor = strtol(buffer, &sobra,
   10);
        /* "sobra" aponta para o primeiro caractere que NAO fez parte do
numero. Se sobra == buffer, nada foi convertido (texto vazio
```

```
ou nao numerico). Se *sobra != '\0', sobrou lixo depois do
numero (ex: "12a"). Nos dois casos, o valor e invalido. */         if
(sobra != buffer && *sobra == '\0') {             return (int) valor;
        }
         printf("  >> Valor invalido. Digite somente
numeros.\n");     }
}

/* ------------------------------------------------------------------------
*  limparTela *  Limpa a tela do terminal para deixar os menus mais
   organizados.
*  Usa o comando certo dependendo do sistema operacional.
*  ------------------------------------------------------------------------
   */void limparTela(void) { #ifdef _WIN32     system("cls"); #else
    system("clear");
#endif
}

/* ------------------------------------------------------------------------
*  pausar
*  Para a execucao ate o usuario apertar ENTER, dando tempo de ler as
*  mensagens na tela antes de voltar para o menu. Descarta qualquer *  coisa
   digitada antes do ENTER, para nao atrapalhar a proxima leitura.
*  ------------------------------------------------------------------------
   */void pausar(void) {     printf("\nPressione ENTER para continuar...");
   int c;     while ((c = getchar()) != '\n' && c != EOF) {         /*
   descarta */
    }
}

/* ------------------------------------------------------------------------
*  obterDataAtual
*  Pega a data atual do relogio do computador e escreve no formato
*  DD/MM/AAAA dentro de "destino". Usada para preencher automaticamente
*  as datas de emprestimo e devolucao, sem depender de o usuario *  digitar a
   data certa.
*  ------------------------------------------------------------------------
   */void obterDataAtual(char *destino) {     time_t agora = time(NULL);
   struct tm *tempoLocal = localtime(&agora);     strftime(destino, 11,
   "%d/%m/%Y", tempoLocal); }


/* ========================================================================
*  FUNCOES DE BUSCA *  Percorrem o vetor procurando um registro com o codigo
   informado.
*  Retornam a POSICAO (indice) do registro no vetor, ou -1 se nao achar.
*  Retornar a posicao (em vez do registro inteiro) permite tanto LER *  quanto
   ALTERAR o registro original a partir do vetor global.
*  ========================================================================
   */int buscarAutorPorCodigo(int codigo) {     int i;     for (i = 0; i <
   totalAutores; i++) {         if (autores[i].codigo == codigo) {
            return i;
}
    }     return -1; }  int
buscarEditoraPorCodigo(int codigo) {
int i;     for (i = 0; i < totalEditoras;
i++) {         if (editoras[i].codigo ==
codigo) {             return i;         }
```

```
    }     return -1; }  int
buscarLivroPorCodigo(int codigo) {
int i;     for (i = 0; i < totalLivros;
i++) {         if (livros[i].codigo ==
codigo) {             return i;         }
    }     return -1; }  int
buscarUsuarioPorCodigo(int codigo) {
int i;     for (i = 0; i < totalUsuarios;
i++) {         if (usuarios[i].codigo ==
codigo) {             return i;         }
    }
return -1; }


/* ========================================================================
*  FUNCOES DE EXIBICAO (tabelas)
*  Apenas imprimem os dados na tela, sem pausar. Sao reaproveitadas
*  tanto pelas telas de "consultar" quanto pelas de "alterar/excluir" *  (que
   precisam mostrar a lista antes de pedir um codigo).
*  ========================================================================
   */void exibirAutores(void) {     if (totalAutores == 0) {
   printf("Nenhum autor cadastrado.\n");         return;     }     printf("%-6s
   %-30s %-20s\n", "Cod.", "Nome", "Nacionalidade");     printf("---------------
   ----------------------------------------\n");     int i;     for (i = 0; i
   < totalAutores; i++) {         printf("%-6d %-30s %-20s\n",
   autores[i].codigo, autores[i].nome, autores[i].nacionalidade);     }
}  void
exibirEditoras(void) {
if (totalEditoras == 0) {
printf("Nenhuma editora
cadastrada.\n");
return;     }
printf("%-6s %-30s %-
20s\n", "Cod.", "Nome",
"Cidade");     printf("----
--------------------------
--------------------------
\n");     int i;     for (i
= 0; i < totalEditoras;
i++) {         printf("%-6d
%-30s %-20s\n",
editoras[i].codigo,
editoras[i].nome,
editoras[i].cidade);     }
}  void exibirLivros(void) {     if (totalLivros == 0) {         printf("Nenhum
livro cadastrado.\n");         return;     }     printf("%-6s %-30s %-6s %-6s %-
6s\n", "Cod.", "Titulo", "Ano", "Qtd", "Disp.");     printf("----------------------
----------------------------------------\n");     int i;     for (i = 0; i <
totalLivros; i++) {         printf("%-6d %-30s %-6d %-6d %-6d\n",
livros[i].codigo, livros[i].titulo, livros[i].ano,
livros[i].quantidade, livros[i].quantidadeDisponivel);     }
}  void exibirUsuarios(void) {     if (totalUsuarios == 0) {         printf("Nenhum usuario
cadastrado.\n");         return;     }     printf("%-6s %-30s %-15s %-15s\n", "Cod.",
"Nome", "CPF", "Telefone");     printf("-----------------------------------------------------
---------------\n");     int i;     for (i = 0; i < totalUsuarios; i++) {         printf("%-
6d %-30s %-15s %-15s\n",                usuarios[i].codigo, usuarios[i].nome,
usuarios[i].cpf, usuarios[i].telefone);     }
```

```
}  void exibirEmprestimos(void) {     if (totalEmprestimos == 0) {         printf("Nenhum
emprestimo registrado.\n");         return;     }     printf("%-6s %-20s %-20s %-12s %-12s %-
12s\n",            "Cod.", "Usuario", "Livro", "Emprestimo", "Devolucao", "Status");
printf("-------------------------------------------------------------------------------\
n");     int i;     for (i = 0; i < totalEmprestimos; i++) {         int posU =
buscarUsuarioPorCodigo(emprestimos[i].codigoUsuario);         int posL =
buscarLivroPorCodigo(emprestimos[i].codigoLivro);          const char *nomeUsuario = (posU !=
-1) ? usuarios[posU].nome : "(removido)";         const char *tituloLivro = (posL != -1) ?
livros[posL].titulo : "(removido)";
        const char *statusTexto = (emprestimos[i].status == STATUS_EMPRESTADO) ?
"Emprestado" : "Devolvido";         const char *devolucao   =
(strlen(emprestimos[i].dataDevolucao) > 0) ?
emprestimos[i].dataDevolucao : "--";          printf("%-6d %-20s %-20s
%-12s %-12s %-12s\n",                emprestimos[i].codigo,
nomeUsuario, tituloLivro,
emprestimos[i].dataEmprestimo, devolucao, statusTexto);     }
}


/* ============================================================================
*  CRUD - AUTOR
*  ============================================================================ */

/* ----------------------------------------------------------------------
*  cadastrarAutor
*  O codigo do novo autor e gerado automaticamente (maior codigo ja *  usado +
   1), assim o usuario nunca digita nem repete um codigo.
*  ---------------------------------------------------------------------- */void
   cadastrarAutor(void) {     limparTela();     printf("===== CADASTRAR AUTOR
   =====\n\n");      if (totalAutores >= MAX_AUTORES) {         printf("Limite
   maximo de autores atingido!\n");         pausar();         return;     }
     struct Autor novo;     int maior = 0, i;     for (i = 0; i <
totalAutores; i++) {         if (autores[i].codigo > maior) maior
= autores[i].codigo;     }     novo.codigo = maior + 1;
printf("Codigo gerado automaticamente: %d\n\n", novo.codigo);
printf("Nome do autor: ");
    lerLinha(novo.nome, sizeof(novo.nome));      printf("Nacionalidade: ");
lerLinha(novo.nacionalidade, sizeof(novo.nacionalidade));
autores[totalAutores] = novo;   /* copia a struct inteira para o vetor */
totalAutores++;      salvarAutores();                /* grava no arquivo
imediatamente */      printf("\nAutor cadastrado com sucesso!\n");
pausar(); }  void consultarAutores(void) {     limparTela();
printf("===== LISTA DE AUTORES =====\n\n");     exibirAutores();
    pausar();
}

/* ----------------------------------------------------------------------
*  alterarAutor
*  Mostra a lista, pede o codigo do autor a alterar, mostra os dados *
   atuais e pede os novos valores para sobrescrever.
*  ---------------------------------------------------------------------
   */void alterarAutor(void) {     limparTela();     printf("===== ALTERAR
   AUTOR =====\n\n");      if (totalAutores == 0) {         printf("Nenhum
   autor cadastrado.\n");         pausar();         return;     }
     exibirAutores();      int codigo = lerInteiro("\nCodigo do autor a alterar
(0 para cancelar): ");     if (codigo == 0) return;      int pos =
buscarAutorPorCodigo(codigo);     if (pos == -1) {         printf("\nAutor nao
encontrado!\n");         pausar();         return;     }
```

```
     printf("\nDados atuais -> Nome: %s | Nacionalidade: %s\n",
autores[pos].nome, autores[pos].nacionalidade);      printf("Novo nome: ");
lerLinha(autores[pos].nome, sizeof(autores[pos].nome));      printf("Nova
nacionalidade: ");     lerLinha(autores[pos].nacionalidade,
sizeof(autores[pos].nacionalidade));      salvarAutores();      printf("\nAutor
atualizado com sucesso!\n");     pausar(); }

/* ----------------------------------------------------------------------
*  excluirAutor
*  Antes de excluir, verifica se algum Livro usa este autor. Se usar,
*  bloqueia a exclusao para nao deixar um livro "orfao" (apontando *  para
   um autor que nao existe mais).
*  ----------------------------------------------------------------------
   */void excluirAutor(void) {     limparTela();     printf("===== EXCLUIR
   AUTOR =====\n\n");      if (totalAutores == 0) {         printf("Nenhum
   autor cadastrado.\n");         pausar();
        return;
}
     exibirAutores();      int codigo = lerInteiro("\nCodigo do autor a excluir
(0 para cancelar): ");     if (codigo == 0) return;      int pos =
buscarAutorPorCodigo(codigo);     if (pos == -1) {         printf("\nAutor nao
encontrado!\n");         pausar();         return;     }
     int i;     for (i = 0; i < totalLivros; i++) {         if (livros[i].codigoAutor
== codigo) {             printf("\nNao e possivel excluir: existe(m) livro(s)
cadastrado(s) com este autor.\n");             pausar();             return;         }
    }

    /* "Desloca" todos os registros depois da posicao excluida uma casa
para tras, cobrindo o espaco do registro removido. */     for (i = pos;
i < totalAutores - 1; i++) {         autores[i] = autores[i + 1];     }
    totalAutores--;      salvarAutores();
printf("\nAutor excluido com sucesso!\n");
pausar(); }


/* ======================================================================
*  CRUD - EDITORA  (mesma logica do Autor)
*  ======================================================================
   */void cadastrarEditora(void) {     limparTela();     printf("===== CADASTRAR
   EDITORA =====\n\n");      if (totalEditoras >= MAX_EDITORAS) {
   printf("Limite maximo de editoras atingido!\n");         pausar();
   return;     }
     struct Editora nova;     int maior = 0, i;     for (i = 0; i <
totalEditoras; i++) {         if (editoras[i].codigo > maior) maior
= editoras[i].codigo;     }     nova.codigo = maior + 1;
    printf("Codigo gerado automaticamente: %d\n\n", nova.codigo);
printf("Nome da editora: ");     lerLinha(nova.nome,
sizeof(nova.nome));      printf("Cidade: ");
lerLinha(nova.cidade, sizeof(nova.cidade));
editoras[totalEditoras] = nova;     totalEditoras++;
salvarEditoras();      printf("\nEditora cadastrada com
sucesso!\n");     pausar(); }  void consultarEditoras(void) {
limparTela();     printf("===== LISTA DE EDITORAS =====\n\n");
exibirEditoras();     pausar(); }  void alterarEditora(void) {
limparTela();     printf("===== ALTERAR EDITORA =====\n\n");
if (totalEditoras == 0) {         printf("Nenhuma editora
cadastrada.\n");         pausar();         return;     }
```

```
     exibirEditoras();      int codigo = lerInteiro("\nCodigo da editora a
alterar (0 para cancelar): ");     if (codigo == 0) return;      int pos =
buscarEditoraPorCodigo(codigo);     if (pos == -1) {         printf("\nEditora
nao encontrada!\n");         pausar();         return;     }
     printf("\nDados atuais -> Nome: %s | Cidade: %s\n",
editoras[pos].nome, editoras[pos].cidade);      printf("Novo nome: ");
lerLinha(editoras[pos].nome, sizeof(editoras[pos].nome));      printf("Nova
cidade: ");     lerLinha(editoras[pos].cidade,
sizeof(editoras[pos].cidade));      salvarEditoras();
printf("\nEditora atualizada com sucesso!\n");
    pausar(); }  void excluirEditora(void) {
limparTela();     printf("===== EXCLUIR EDITORA
=====\n\n");      if (totalEditoras == 0) {
printf("Nenhuma editora cadastrada.\n");
pausar();         return;     }
     exibirEditoras();      int codigo = lerInteiro("\nCodigo da editora a
excluir (0 para cancelar): ");     if (codigo == 0) return;      int pos =
buscarEditoraPorCodigo(codigo);     if (pos == -1) {         printf("\nEditora
nao encontrada!\n");         pausar();         return;     }
     int i;     for (i = 0; i < totalLivros; i++) {         if (livros[i].codigoEditora
== codigo) {             printf("\nNao e possivel excluir: existe(m) livro(s)
cadastrado(s) com esta editora.\n");             pausar();             return;
}
    }
     for (i = pos; i < totalEditoras - 1; i++)
{         editoras[i] = editoras[i + 1];     }
    totalEditoras--;      salvarEditoras();
printf("\nEditora excluida com sucesso!\n");
pausar(); }


/* ================================================================================
*  CRUD - LIVRO
*  E o mais complexo, porque um Livro se relaciona com um Autor e com *  uma
   Editora (guarda apenas o CODIGO deles, nao os dados inteiros).
*  ================================================================================
   */void cadastrarLivro(void) {     limparTela();     printf("===== CADASTRAR
   LIVRO =====\n\n");      if (totalLivros >= MAX_LIVROS) {
   printf("Limite maximo de livros atingido!\n");
        pausar();         return;     }     if (totalAutores == 0) {
printf("Cadastre pelo menos um autor antes de cadastrar um livro.\n");
pausar();         return;     }     if (totalEditoras == 0) {
printf("Cadastre pelo menos uma editora antes de cadastrar um livro.\n");
pausar();         return;     }
     struct Livro novo;     int maior = 0, i;     for (i = 0; i <
totalLivros; i++) {         if (livros[i].codigo > maior) maior =
livros[i].codigo;     }     novo.codigo = maior + 1;
printf("Codigo gerado automaticamente: %d\n\n", novo.codigo);
printf("Titulo do livro: ");     lerLinha(novo.titulo,
sizeof(novo.titulo));
    /* Mostra os autores cadastrados para o usuario escolher um codigo        valido, em vez
de ter que decorar ou adivinhar o numero. */     printf("\nAutores cadastrados:\n");
exibirAutores();     int codAutor;     while (1) {         codAutor = lerInteiro("\nCodigo do
autor deste livro (0 para cancelar o cadastro): ");         if (codAutor == 0) {
printf("\nCadastro cancelado.\n");             pausar();             return;         }
if (buscarAutorPorCodigo(codAutor) != -1) break;
        printf("  >> Codigo de autor invalido, tente novamente.\n");     }
novo.codigoAutor = codAutor;      printf("\nEditoras cadastradas:\n");     exibirEditoras();
```

```
int codEditora;     while (1) {         codEditora = lerInteiro("\nCodigo da editora deste
livro (0 para cancelar o cadastro):
");         if (codEditora == 0) {             printf("\nCadastro
cancelado.\n");             pausar();             return;         }
if (buscarEditoraPorCodigo(codEditora) != -1) break;         printf("
>> Codigo de editora invalido, tente novamente.\n");     }
novo.codigoEditora = codEditora;

    novo.ano = lerInteiro("Ano de publicacao: ");      int qtd;     do {
qtd = lerInteiro("Quantidade de exemplares: ");         if (qtd <= 0)
printf("  >> A quantidade deve ser maior que zero.\n");     } while (qtd <=
0);
     novo.quantidade = qtd;     novo.quantidadeDisponivel = qtd;  /* no cadastro, nenhum
exemplar esta emprestado ainda */      livros[totalLivros] = novo;     totalLivros++;
salvarLivros();      printf("\nLivro cadastrado com sucesso!\n");     pausar(); }

/* ----------------------------------------------------------------------
*  consultarLivros
*  Alem de listar todos os livros, permite ver os detalhes de um livro
*  especifico, ja "traduzindo" o codigoAutor/codigoEditora para o nome
*  de verdade (isso e o "relacionamento" entre as structs mencionado *  no
   planejamento do projeto).
*  ---------------------------------------------------------------------- */void
   consultarLivros(void) {     limparTela();     printf("===== LISTA DE LIVROS
   =====\n\n");     exibirLivros();      if (totalLivros == 0) {
   pausar();         return;     }
     int codigo = lerInteiro("\nCodigo de um livro para ver detalhes (0 para voltar): ");
if (codigo == 0) return;      int pos = buscarLivroPorCodigo(codigo);     if (pos == -1) {
printf("\nLivro nao encontrado!\n");     } else {         int posAutor =
buscarAutorPorCodigo(livros[pos].codigoAutor);         int posEditora =
buscarEditoraPorCodigo(livros[pos].codigoEditora);          printf("\n----- DETALHES DO
LIVRO -----\n");         printf("Codigo.......: %d\n", livros[pos].codigo);
printf("Titulo.......: %s\n", livros[pos].titulo);         printf("Autor........: %s\n",
(posAutor != -1) ? autores[posAutor].nome : "(nao encontrado)");
printf("Editora......: %s\n", (posEditora != -1) ? editoras[posEditora].nome : "(nao
encontrada)");         printf("Ano..........: %d\n", livros[pos].ano);
printf("Exemplares...: %d (disponiveis: %d)\n", livros[pos].quantidade,
livros[pos].quantidadeDisponivel);
    }
    pausar();
}

/* ----------------------------------------------------------------------
*  alterarLivro
*  Ao mudar a quantidade TOTAL de exemplares, a quantidade DISPONIVEL
*  e ajustada pela diferenca, sem nunca ficar negativa nem maior que *  o novo
   total (protege a consistencia dos dados).
*  ---------------------------------------------------------------------- */
   */void alterarLivro(void) {     limparTela();     printf("===== ALTERAR
   LIVRO =====\n\n");      if (totalLivros == 0) {         printf("Nenhum
   livro cadastrado.\n");         pausar();         return;     }
     exibirLivros();      int codigo = lerInteiro("\nCodigo do livro a alterar
(0 para cancelar): ");     if (codigo == 0) return;      int pos =
buscarLivroPorCodigo(codigo);     if (pos == -1) {         printf("\nLivro nao
encontrado!\n");         pausar();         return;     }
     printf("\nTitulo atual: %s\n", livros[pos].titulo);     printf("Novo
titulo: ");     lerLinha(livros[pos].titulo, sizeof(livros[pos].titulo));
printf("\nAno atual: %d\n", livros[pos].ano);     livros[pos].ano =
```

```
lerInteiro("Novo ano: ");      printf("\nQuantidade atual: %d (disponiveis:
%d)\n", livros[pos].quantidade, livros[pos].quantidadeDisponivel);     int
novaQtd;     do {         novaQtd = lerInteiro("Nova quantidade total de
exemplares: ");
        if (novaQtd <= 0) printf("  >> A quantidade deve ser maior que zero.\n");
} while (novaQtd <= 0);
     int diferenca = novaQtd - livros[pos].quantidade;
livros[pos].quantidadeDisponivel += diferenca;     if
(livros[pos].quantidadeDisponivel < 0) livros[pos].quantidadeDisponivel = 0;     if
(livros[pos].quantidadeDisponivel > novaQtd) livros[pos].quantidadeDisponivel =
novaQtd;     livros[pos].quantidade = novaQtd;      salvarLivros();
printf("\nLivro atualizado com sucesso!\n");     pausar();
}

/* ----------------------------------------------------------------------
*  excluirLivro
*  Bloqueia a exclusao se o livro tiver algum emprestimo em aberto
*  (status == STATUS_EMPRESTADO), para nao perder o historico de um *  livro
   que ainda esta com um usuario.
*  ----------------------------------------------------------------------
   */void excluirLivro(void) {     limparTela();     printf("===== EXCLUIR
   LIVRO =====\n\n");      if (totalLivros == 0) {         printf("Nenhum
   livro cadastrado.\n");         pausar();         return;     }
     exibirLivros();      int codigo = lerInteiro("\nCodigo do livro a excluir
(0 para cancelar): ");     if (codigo == 0) return;      int pos =
buscarLivroPorCodigo(codigo);     if (pos == -1) {         printf("\nLivro nao
encontrado!\n");         pausar();         return;     }
     int i;     for (i = 0; i < totalEmprestimos; i++) {         if
(emprestimos[i].codigoLivro == codigo && emprestimos[i].status == STATUS_EMPRESTADO) {
printf("\nNao e possivel excluir: este livro possui emprestimo(s) em aberto.\n");
pausar();             return;         }
    }
     for (i = pos; i < totalLivros - 1; i++)
{         livros[i] = livros[i + 1];     }
    totalLivros--;      salvarLivros();
printf("\nLivro excluido com sucesso!\n");
pausar(); }


/* ======================================================================
*  CRUD - USUARIO  (mesma logica do Autor/Editora)
*  ======================================================================
   */void cadastrarUsuario(void) {     limparTela();     printf("===== CADASTRAR
   USUARIO =====\n\n");

    if (totalUsuarios >= MAX_USUARIOS) {
printf("Limite maximo de usuarios atingido!\n");
pausar();         return;     }
     struct Usuario novo;     int maior = 0, i;     for (i = 0; i <
totalUsuarios; i++) {         if (usuarios[i].codigo > maior) maior
= usuarios[i].codigo;     }     novo.codigo = maior + 1;
printf("Codigo gerado automaticamente: %d\n\n", novo.codigo);
printf("Nome: ");     lerLinha(novo.nome, sizeof(novo.nome));
printf("CPF: ");     lerLinha(novo.cpf, sizeof(novo.cpf));
printf("Telefone: ");     lerLinha(novo.telefone,
sizeof(novo.telefone));      usuarios[totalUsuarios] = novo;
totalUsuarios++;      salvarUsuarios();      printf("\nUsuario
cadastrado com sucesso!\n");     pausar(); }  void
```

```
consultarUsuarios(void) {     limparTela();     printf("===== LISTA
DE USUARIOS =====\n\n");     exibirUsuarios();     pausar(); }
void alterarUsuario(void) {     limparTela();     printf("=====
ALTERAR USUARIO =====\n\n");      if (totalUsuarios == 0) {
printf("Nenhum usuario cadastrado.\n");         pausar();
return;     }
     exibirUsuarios();      int codigo = lerInteiro("\nCodigo do usuario a
alterar (0 para cancelar): ");     if (codigo == 0) return;      int pos =
buscarUsuarioPorCodigo(codigo);     if (pos == -1) {         printf("\nUsuario
nao encontrado!\n");         pausar();
        return;
}
     printf("\nDados atuais -> Nome: %s | CPF: %s | Telefone: %s\n",
usuarios[pos].nome, usuarios[pos].cpf, usuarios[pos].telefone);
printf("Novo nome: ");     lerLinha(usuarios[pos].nome,
sizeof(usuarios[pos].nome));      printf("Novo CPF: ");
lerLinha(usuarios[pos].cpf, sizeof(usuarios[pos].cpf));      printf("Novo
telefone: ");     lerLinha(usuarios[pos].telefone,
sizeof(usuarios[pos].telefone));      salvarUsuarios();
printf("\nUsuario atualizado com sucesso!\n");     pausar(); }  void
excluirUsuario(void) {     limparTela();     printf("===== EXCLUIR USUARIO
=====\n\n");      if (totalUsuarios == 0) {         printf("Nenhum usuario
cadastrado.\n");         pausar();         return;     }
     exibirUsuarios();      int codigo = lerInteiro("\nCodigo do usuario a
excluir (0 para cancelar): ");     if (codigo == 0) return;      int pos =
buscarUsuarioPorCodigo(codigo);     if (pos == -1) {         printf("\nUsuario
nao encontrado!\n");         pausar();         return;     }
     int i;     for (i = 0; i < totalEmprestimos; i++) {         if
(emprestimos[i].codigoUsuario == codigo && emprestimos[i].status == STATUS_EMPRESTADO) {
printf("\nNao e possivel excluir: este usuario possui emprestimo(s) em aberto.\ n");
pausar();             return;         }
    }
     for (i = pos; i < totalUsuarios - 1; i++)
{         usuarios[i] = usuarios[i + 1];     }
totalUsuarios--;
     salvarUsuarios();      printf("\nUsuario
excluido com sucesso!\n");     pausar(); }


/* ========================================================================
*  EMPRESTIMOS E DEVOLUCOES
*  Este e o modulo que "liga" Usuario e Livro. Ele e responsavel por
*  verificar a disponibilidade e atualizar a quantidadeDisponivel do *  livro
   automaticamente, como pedido no planejamento do projeto.
*  ========================================================================
   */void consultarEmprestimos(void) {     limparTela();     printf("===== LISTA
   DE EMPRESTIMOS =====\n\n");     exibirEmprestimos();     pausar(); }

/* ------------------------------------------------------------------
*  registrarEmprestimo *  So permite o emprestimo se existir pelo menos 1
   exemplar disponivel.
*  A data do emprestimo e preenchida sozinha com a data de hoje.
*  ------------------------------------------------------------------ */void
   registrarEmprestimo(void) {     limparTela();     printf("===== REGISTRAR
   EMPRESTIMO =====\n\n");      if (totalEmprestimos >= MAX_EMPRESTIMOS) {
   printf("Limite maximo de emprestimos atingido!\n");         pausar();
   return;     }     if (totalUsuarios == 0) {         printf("Nenhum usuario
   cadastrado.\n");         pausar();         return;     }     if (totalLivros
```

```
   == 0) {         printf("Nenhum livro cadastrado.\n");         pausar();
   return;     }
     printf("Usuarios cadastrados:\n");     exibirUsuarios();     int
codUsuario;     while (1) {         codUsuario = lerInteiro("\nCodigo do
usuario (0 para cancelar): ");         if (codUsuario == 0) {
printf("\nOperacao cancelada.\n");             pausar();
return;         }         if (buscarUsuarioPorCodigo(codUsuario) != -1)
break;         printf("  >> Usuario nao encontrado, tente novamente.\n");
    }
     printf("\nLivros cadastrados:\n");     exibirLivros();     int codLivro,
posLivro = -1;     while (1) {         codLivro = lerInteiro("\nCodigo do livro (0
para cancelar): ");         if (codLivro == 0) {             printf("\nOperacao
cancelada.\n");             pausar();             return;         }
posLivro = buscarLivroPorCodigo(codLivro);         if (posLivro == -1) {
printf("  >> Livro nao encontrado, tente novamente.\n");             continue;
}         if (livros[posLivro].quantidadeDisponivel <= 0) {             printf("
>> Nao ha exemplares disponiveis deste livro no momento.\n");             continue;
}         break;     }
     struct Emprestimo novo;     int maior = 0, i;     for (i = 0; i <
totalEmprestimos; i++) {         if (emprestimos[i].codigo > maior) maior =
emprestimos[i].codigo;     }     novo.codigo = maior + 1;     novo.codigoUsuario =
codUsuario;     novo.codigoLivro = codLivro;
obterDataAtual(novo.dataEmprestimo);   /* data de hoje, automatica */
novo.dataDevolucao[0] = '\0';          /* ainda nao foi devolvido  */
novo.status = STATUS_EMPRESTADO;      emprestimos[totalEmprestimos] = novo;
totalEmprestimos++;      livros[posLivro].quantidadeDisponivel--;   /* um exemplar
a menos disponivel */      salvarEmprestimos();     salvarLivros();
printf("\nEmprestimo numero %d registrado com sucesso em %s!\n", novo.codigo,
novo.dataEmprestimo);     pausar(); }

/* ----------------------------------------------------------------------
*  registrarDevolucao
*  Marca um emprestimo em aberto como devolvido (preenchendo a data de *
   hoje automaticamente) e devolve o exemplar ao estoque disponivel.
*  ----------------------------------------------------------------------
   */void registrarDevolucao(void) {     limparTela();     printf("=====
   REGISTRAR DEVOLUCAO =====\n\n");

    if (totalEmprestimos == 0) {
printf("Nenhum emprestimo registrado.\n");
pausar();         return;     }
     printf("Emprestimos em aberto:\n\n");     int existeAberto = 0, i;
for (i = 0; i < totalEmprestimos; i++) {         if
(emprestimos[i].status == STATUS_EMPRESTADO) {             int posL =
buscarLivroPorCodigo(emprestimos[i].codigoLivro);
printf("Codigo %d - Livro: %s (emprestado em %s)\n",
emprestimos[i].codigo,                     (posL != -1) ?
livros[posL].titulo : "(removido)",
emprestimos[i].dataEmprestimo);             existeAberto = 1;         }
    }
     if (!existeAberto) {         printf("Nao ha emprestimos
em aberto no momento.\n");         pausar();         return;
}
     int codigo = lerInteiro("\nCodigo do emprestimo a devolver (0 para cancelar):
");     if (codigo == 0) return;      int pos = -1;     for (i = 0; i <
totalEmprestimos; i++) {         if (emprestimos[i].codigo == codigo) {
pos = i;             break;         }
    }
```

```
    if (pos == -1) {         printf("\nEmprestimo nao
encontrado!\n");         pausar();         return;     }     if
(emprestimos[pos].status == STATUS_DEVOLVIDO) {         printf("\nEste
emprestimo ja foi devolvido anteriormente.\n");         pausar();
return;     }
    obterDataAtual(emprestimos[pos].dataDevolucao);     emprestimos[pos].status =
STATUS_DEVOLVIDO;      int posLivro =
buscarLivroPorCodigo(emprestimos[pos].codigoLivro);     if (posLivro != -1) {
livros[posLivro].quantidadeDisponivel++;         if
(livros[posLivro].quantidadeDisponivel > livros[posLivro].quantidade) {
livros[posLivro].quantidadeDisponivel = livros[posLivro].quantidade;         }
    }

    salvarEmprestimos();     salvarLivros();      printf("\nDevolucao registrada com
sucesso em %s!\n", emprestimos[pos].dataDevolucao);     pausar(); }


/* ========================================================================
*  PERSISTENCIA EM ARQUIVO
*  Cada par salvar/carregar grava (ou le) primeiro a quantidade de
*  registros (um int) e depois o vetor inteiro de structs, em modo
*  binario ("wb"/"rb"). Assim o proximo carregamento sabe exatamente *  quantos
   registros ler de volta.
*  ========================================================================
   */void salvarAutores(void) {     FILE *arq = fopen(ARQ_AUTORES, "wb");     if
   (arq == NULL) { printf("Erro ao salvar autores!\n"); return; }
   fwrite(&totalAutores, sizeof(int), 1, arq);     fwrite(autores, sizeof(struct
   Autor), totalAutores, arq);     fclose(arq); }  void carregarAutores(void) {
   FILE *arq = fopen(ARQ_AUTORES, "rb");     if (arq == NULL) { totalAutores =
   0; return; }   /* arquivo ainda nao existe */     fread(&totalAutores,
   sizeof(int), 1, arq);     if (totalAutores < 0 || totalAutores > MAX_AUTORES)
   totalAutores = 0;     else fread(autores, sizeof(struct Autor), totalAutores,
   arq);     fclose(arq); }  void salvarEditoras(void) {     FILE *arq =
   fopen(ARQ_EDITORAS, "wb");     if (arq == NULL) { printf("Erro ao salvar
   editoras!\n"); return; }     fwrite(&totalEditoras, sizeof(int), 1, arq);
   fwrite(editoras, sizeof(struct Editora), totalEditoras, arq);
   fclose(arq); }  void carregarEditoras(void) {     FILE *arq =
   fopen(ARQ_EDITORAS, "rb");     if (arq == NULL) { totalEditoras = 0; return;
   }     fread(&totalEditoras, sizeof(int), 1, arq);     if (totalEditoras < 0
   || totalEditoras > MAX_EDITORAS) totalEditoras = 0;     else fread(editoras,
   sizeof(struct Editora), totalEditoras, arq);     fclose(arq); }  void
   salvarLivros(void) {     FILE *arq = fopen(ARQ_LIVROS, "wb");     if (arq ==
   NULL) { printf("Erro ao salvar livros!\n"); return; }
   fwrite(&totalLivros, sizeof(int), 1, arq);     fwrite(livros, sizeof(struct
   Livro), totalLivros, arq);     fclose(arq); }  void carregarLivros(void) {
    FILE *arq = fopen(ARQ_LIVROS, "rb");     if (arq == NULL) { totalLivros = 0; return;
}     fread(&totalLivros, sizeof(int), 1, arq);     if (totalLivros < 0 || totalLivros >
MAX_LIVROS) totalLivros = 0;     else fread(livros, sizeof(struct Livro), totalLivros,
arq);     fclose(arq); }  void salvarUsuarios(void) {     FILE *arq = fopen(ARQ_USUARIOS,
"wb");     if (arq == NULL) { printf("Erro ao salvar usuarios!\n"); return; }
fwrite(&totalUsuarios, sizeof(int), 1, arq);     fwrite(usuarios, sizeof(struct Usuario),
totalUsuarios, arq);     fclose(arq); }  void carregarUsuarios(void) {     FILE *arq =
fopen(ARQ_USUARIOS, "rb");     if (arq == NULL) { totalUsuarios = 0; return; }
fread(&totalUsuarios, sizeof(int), 1, arq);     if (totalUsuarios < 0 || totalUsuarios >
MAX_USUARIOS) totalUsuarios = 0;     else fread(usuarios, sizeof(struct Usuario),
totalUsuarios, arq);     fclose(arq); }  void salvarEmprestimos(void) {     FILE *arq =
fopen(ARQ_EMPRESTIMOS, "wb");     if (arq == NULL) { printf("Erro ao salvar
emprestimos!\n"); return; }     fwrite(&totalEmprestimos, sizeof(int), 1, arq);
```

```
fwrite(emprestimos, sizeof(struct Emprestimo), totalEmprestimos, arq);     fclose(arq); }
void carregarEmprestimos(void) {     FILE *arq = fopen(ARQ_EMPRESTIMOS, "rb");     if
(arq == NULL) { totalEmprestimos = 0; return; }     fread(&totalEmprestimos, sizeof(int),
1, arq);     if (totalEmprestimos < 0 || totalEmprestimos > MAX_EMPRESTIMOS)
totalEmprestimos = 0;     else fread(emprestimos, sizeof(struct Emprestimo),
totalEmprestimos, arq);     fclose(arq); }

/* Chama todos os "carregar" de uma vez so, usada no inicio do programa */
void carregarTudo(void) {     carregarAutores();     carregarEditoras();
carregarLivros();     carregarUsuarios();     carregarEmprestimos(); }


/* ================================================================================
*  MENUS
*  Cada menu segue o mesmo padrao: um "do...while" que mostra as
*  opcoes, le a escolha do usuario com lerInteiro, e usa um "switch"
*  para chamar a funcao correspondente. O laco so termina quando o *  usuario
   digita 0 (Voltar/Sair).
*  ================================================================================
   */void menuPrincipal(void) {     int opcao;     do {         limparTela();
   printf("========================================================\n");
   printf("     SISTEMA DE GERENCIAMENTO DE BIBLIOTECA\n");
   printf("========================================================\n\n");
   printf("  1 - Gerenciar Livros\n");         printf("  2 - Gerenciar
   Autores\n");         printf("  3 - Gerenciar Editoras\n");         printf("
   4 - Gerenciar Usuarios\n");         printf("  5 - Gerenciar Emprestimos e
   Devolucoes\n");         printf("  0 - Sair\n");         printf("-------------
   ----------------------------------------\n");          opcao =
   lerInteiro("Escolha uma opcao: ");          switch (opcao) {             case
   1: menuLivros();      break;             case 2: menuAutores();     break;
   case 3: menuEditoras();    break;             case 4: menuUsuarios();
   break;             case 5: menuEmprestimos(); break;             case 0:
   printf("\nEncerrando o sistema...\n"); break;             default:
               printf("\nOpcao invalida!\n");
pausar();         }
    } while (opcao != 0);
}  void menuLivros(void) {     int opcao;     do {         limparTela();
printf("===== GERENCIAR LIVROS =====\n\n");         printf("  1 - Cadastrar
Livro\n");         printf("  2 - Consultar Livros\n");         printf("  3
- Alterar Livro\n");         printf("  4 - Excluir Livro\n");
printf("  0 - Voltar ao menu principal\n");         printf("---------------
---------------------------------------\n");          opcao =
lerInteiro("Escolha uma opcao: ");          switch (opcao) {
case 1: cadastrarLivro(); break;             case 2: consultarLivros();
break;             case 3: alterarLivro(); break;             case 4:
excluirLivro(); break;             case 0: break;             default:
               printf("\nOpcao invalida!\n");
pausar();         }
    } while (opcao != 0);
}
 void menuAutores(void) {     int opcao;     do {         limparTela();
printf("===== GERENCIAR AUTORES =====\n\n");         printf("  1 -
Cadastrar Autor\n");         printf("  2 - Consultar Autores\n");
printf("  3 - Alterar Autor\n");         printf("  4 - Excluir Autor\n");
printf("  0 - Voltar ao menu principal\n");         printf("---------------
---------------------------------------\n");          opcao =
lerInteiro("Escolha uma opcao: ");          switch (opcao) {
case 1: cadastrarAutor(); break;             case 2: consultarAutores();
```

```
break;             case 3: alterarAutor(); break;             case 4:
excluirAutor(); break;             case 0: break;             default:
                printf("\nOpcao invalida!\n");
pausar();          }
    } while (opcao != 0);
}  void menuEditoras(void) {     int opcao;     do {         limparTela();
printf("===== GERENCIAR EDITORAS =====\n\n");         printf("  1 -
Cadastrar Editora\n");         printf("  2 - Consultar Editoras\n");
printf("  3 - Alterar Editora\n");         printf("  4 - Excluir
Editora\n");         printf("  0 - Voltar ao menu principal\n");
printf("-----------------------------------------------------\n");
opcao = lerInteiro("Escolha uma opcao: ");          switch (opcao) {
case 1: cadastrarEditora(); break;             case 2: consultarEditoras();
break;             case 3: alterarEditora(); break;             case 4:
excluirEditora(); break;             case 0: break;             default:
                printf("\nOpcao invalida!\n");
pausar();          }
    } while (opcao != 0);
}  void
menuUsuarios(void) {
int opcao;     do {
        limparTela();         printf("===== GERENCIAR USUARIOS =====\n\n");
printf("  1 - Cadastrar Usuario\n");         printf("  2 - Consultar
Usuarios\n");         printf("  3 - Alterar Usuario\n");         printf("
4 - Excluir Usuario\n");         printf("  0 - Voltar ao menu
principal\n");         printf("-----------------------------------------
----------\n");          opcao = lerInteiro("Escolha uma opcao: ");
switch (opcao) {             case 1: cadastrarUsuario(); break;
case 2: consultarUsuarios(); break;             case 3: alterarUsuario();
break;             case 4: excluirUsuario(); break;             case 0:
break;             default:
                printf("\nOpcao invalida!\n");
pausar();          }
    } while (opcao != 0);
}  void menuEmprestimos(void) {     int opcao;     do {
limparTela();         printf("===== GERENCIAR EMPRESTIMOS E DEVOLUCOES
=====\n\n");         printf("  1 - Registrar Emprestimo\n");
printf("  2 - Registrar Devolucao\n");         printf("  3 - Consultar
Emprestimos\n");         printf("  0 - Voltar ao menu principal\n");
printf("-----------------------------------------------------\n");
opcao = lerInteiro("Escolha uma opcao: ");          switch (opcao) {
case 1: registrarEmprestimo(); break;             case 2:
registrarDevolucao(); break;             case 3: consultarEmprestimos();
break;             case 0: break;             default:
                printf("\nOpcao invalida!\n");
pausar();          }
    } while (opcao != 0);
}
```